算法设计与分析（第五讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP，NP 完全性，近似算法

从哈希出发的衍生算法

- 近似成员查询/测试
- 流式统计
- 相似性与检索

成员测试问题

- **问题：**存储一个超大集合U的子集S
- 例如
  - U是包含所有 128bit 字符串的集合，$|U| = 2^{128}$
  - S是一些想要存储的字符串，$|S| \approx 100{,}000$

- **目标**
  - 成员测试：快速确认一个字符串$x \in S$?
  - 使存储空间尽可能的小

用哈希表解决

- **思路：**将集合S中的元素映射到大小为m的哈希表A上，利用哈希函数h
- **成员测试：**$x \in S \Rightarrow A[h(x)] = x$
- **存储大小：**$m * |x|$

- **解决冲突：**链地址法，开放寻址法
- **缺点：理想状态**下就需要$m * |x|$存储空间，$|x|$可能很大，比如基因序列

为什么需要新的数据结构?

应用：路由器过滤恶意的IP地址

- 集合的可能范围非常大
- 集合中元素的大小较大（ipv6, 128位）
- 哈希表存储在磁盘上，查找操作代价高
- 大多数查找操作都是不成功的查找

- **在进行查找时浪费了大量的时间和空间**

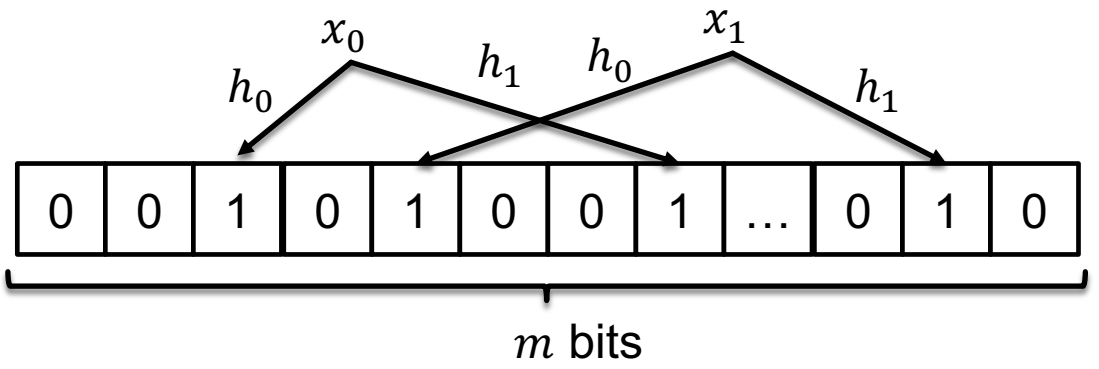
布隆过滤器

- 与哈希表关系密切
- 常数的时间复杂度
- 极其节省空间
- 可以并行计算
- 基于概率的数据结构：**假阳性**

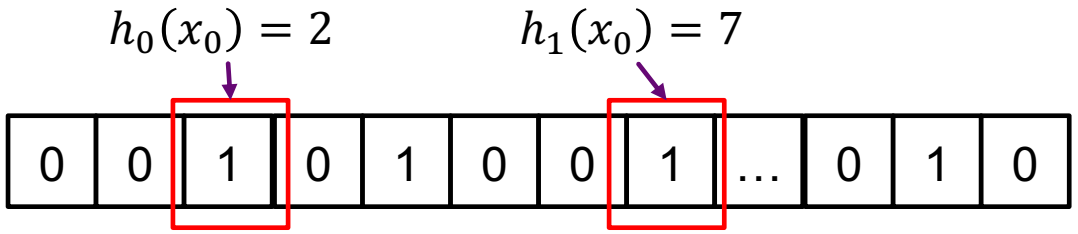
布隆过滤器

- 用k个哈希函数将n个元素映射到大小为m的二进制数组中

构建
(k=2)



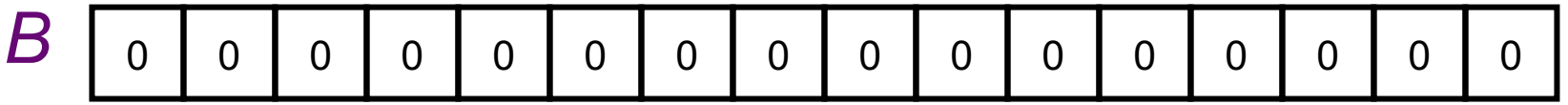
测试
(k=2)



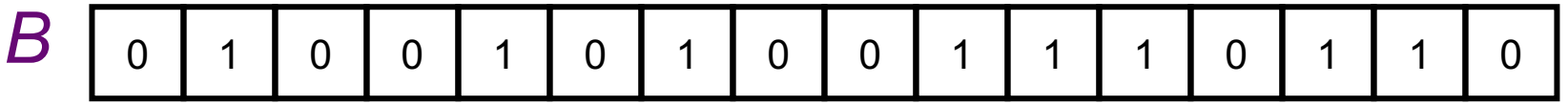
如果都为1，则在该成员存在

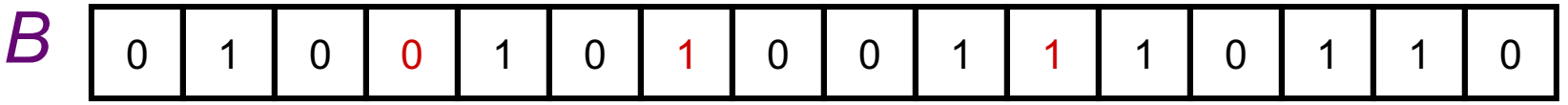
布隆过滤器

从一个长度为 m 的二进制数组B开始，全部初始化为 0

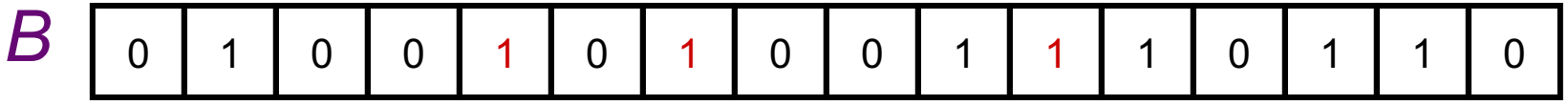


对集合S中的每个元素x_j进行k次哈希。如果$h_i(x_j) = a$则将B的第a位设置为1





可能存在假阳性：所有k值对应的位置都是1，但y不在集合S



n 个元素， *m = cn* 比特， *k* 个哈希函数，假阳性概率*P*

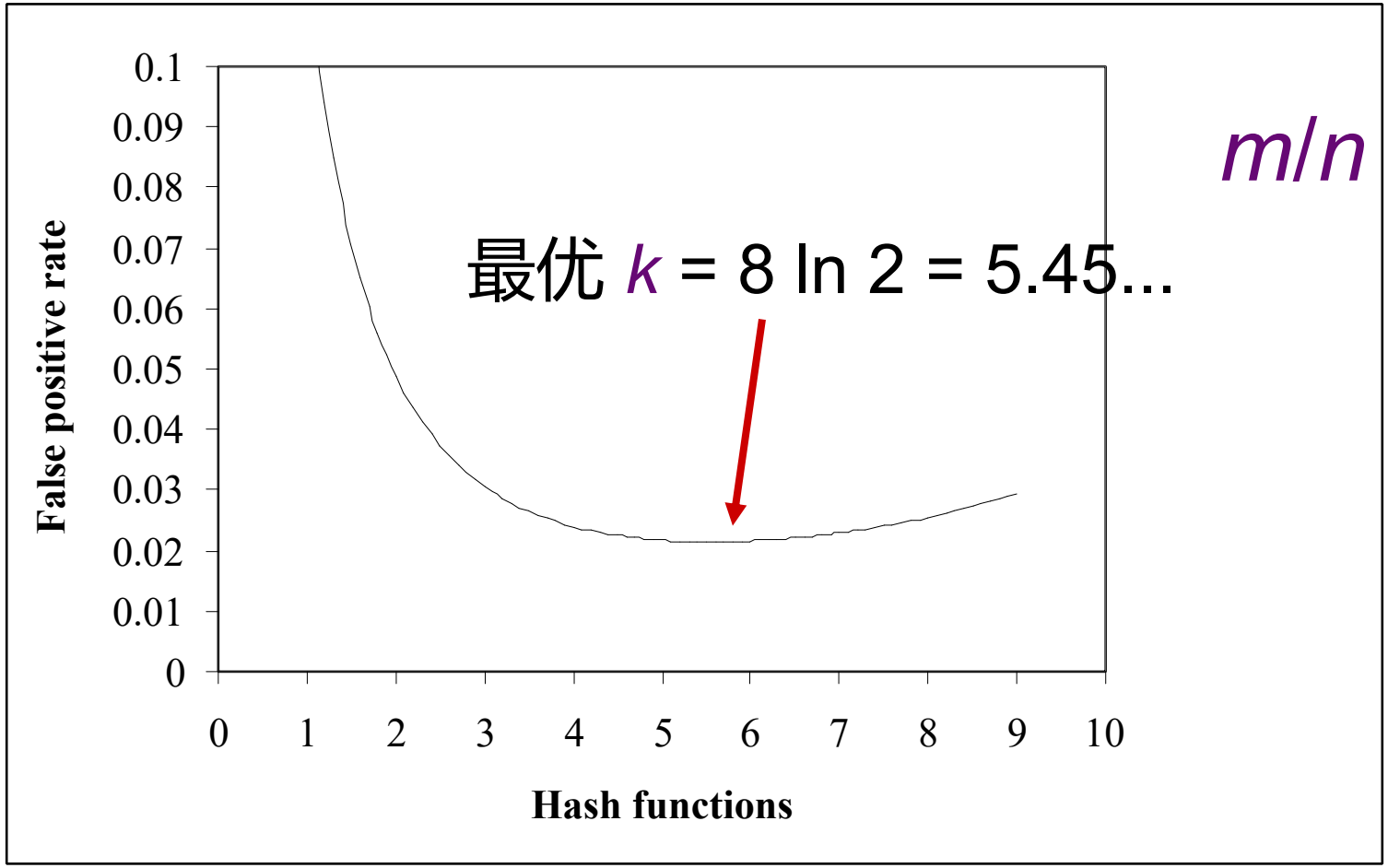
假阳性概率分析

- n个元素经过 k个哈希函数插入，某个位不被设置为 1 的概率为：$\left(1-\frac{1}{m}\right)^{kn}$
- 某个位已经被设置为 1 的概率是： $1-\left(1-\frac{1}{m}\right)^{kn}$
- 不在集合中的元素在所有k个哈希函数映射的位置上都被设置为1的概率： $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)^{k}$
- 泰勒展开的近似：$e^{-x} \approx 1-x \Rightarrow 1-\frac{1}{m}=e^{-\frac{1}{m}}$
- 假阳性概率近似为：$\mathrm{P}=\left(1-e^{-\frac{kn}{m}}\right)^{k}$

其他参数

- 最优的哈希函数个数：$\frac{d}{dk}P = 0 \Rightarrow k = \frac{m}{n}\ln 2$
- 数组大小：$m = -\frac{n \ln P}{(\ln 2)^2}$
- 给定哈希函数个数和数组大小，可以插入的最多元素数：$n = \frac{m \ln 2}{k}$
- 单位空间复杂度：$\frac{m}{n} \approx 1.44 \log_2(\frac{1}{P})$

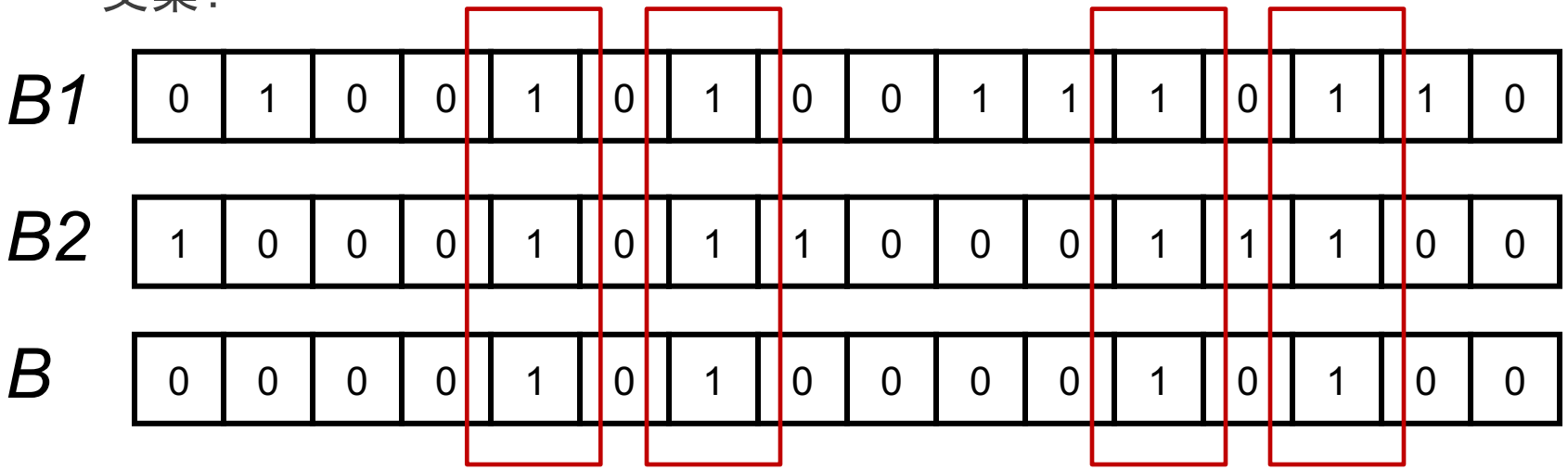
例子


0.1
0.09
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0
False positive rate
0 1 2 3 4 5 6 7 8 9 10
Hash functions
最优 k = 8 ln 2 = 5.45...

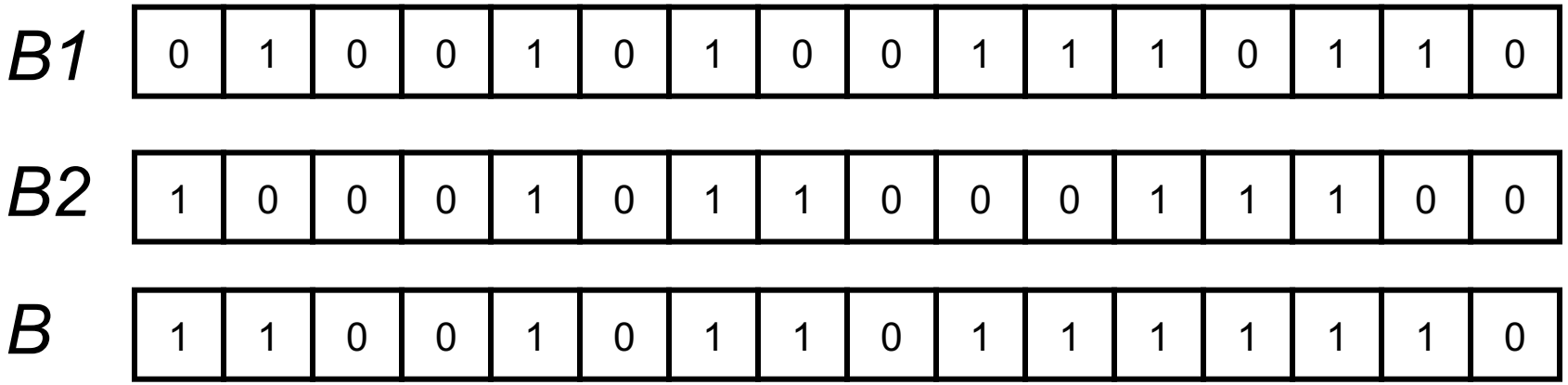
$m/n = 8$

布隆过滤器的计算

- 交集?

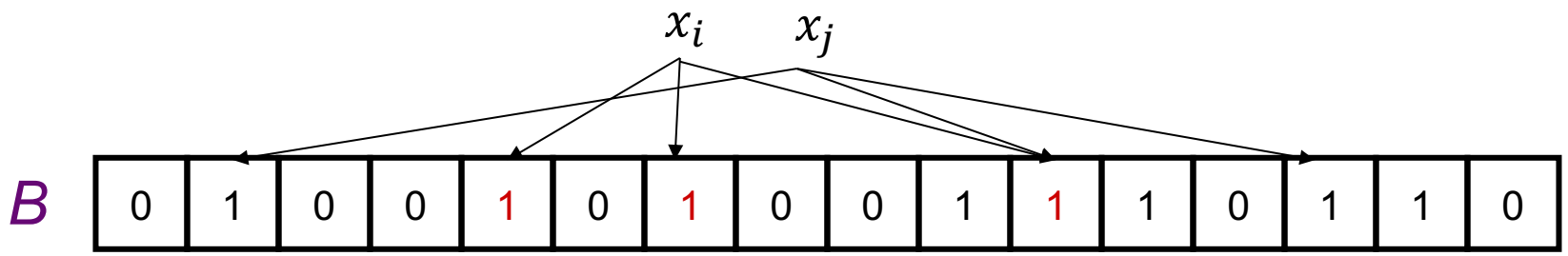


- 并集?



删除操作

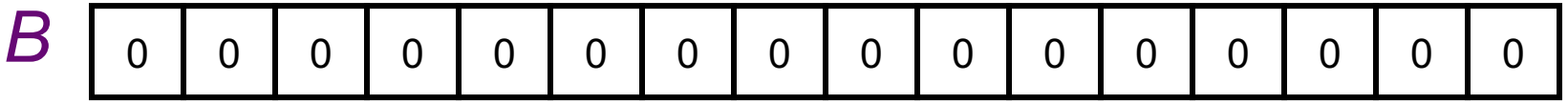
- 布隆过滤器可以进行插入，但删除困难



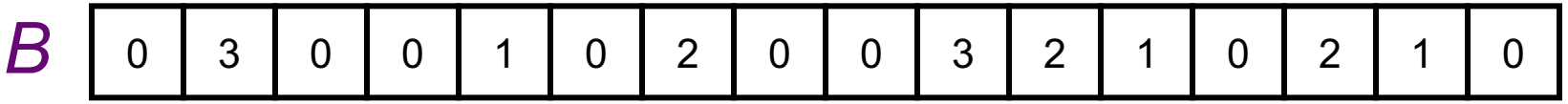
- 删除x_i：需要把x_i对应的映射位置置为0，但这样做会影响x_j(其他元素)

计数布隆过滤器

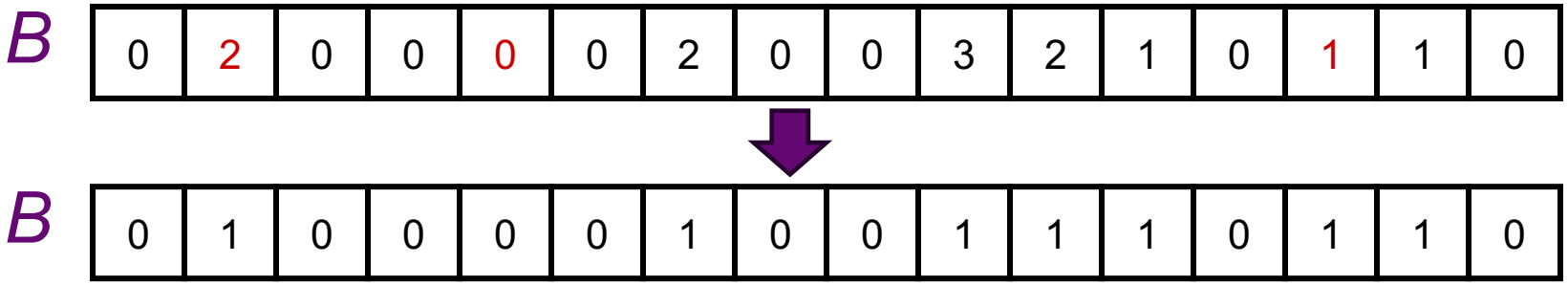
从一个长度为 m 的二进制数组B开始，全部初始化为 0



对集合S中的每个元素x_j进行k次哈希。如果$h_i(x_j) = a$则将B的第a位+1



删除x_j，将x_j哈希映射的位置-1



溢出问题：每一个计数器都需要足够大防止溢出

计数布隆过滤器理论分析

- **假设**哈希函数会将插入的元素均匀映射到位数组中，使得每个计数器的负载遵循**泊松分布**。可以通过**泊松分布**来估算**计数器上限**。
- 每个计数器的平均负载：$\lambda = \frac{kn}{m}$
- 根据泊松分布，计数器的负载概率可以表示为：

$$P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

- 例如λ=2，4比特计数器，

$$P(X \geq 15) = 1 - \sum_{x=0}^{14} \frac{\lambda^x e^{-\lambda}}{x!}$$

成员测试数据结构

数据结构	高性能	高空间利用率	删除支持
Bloom Filter	✓	✓	✗
Counting Bloom Filter	✓	✗	✓

从布谷鸟哈希到布谷鸟过滤器

- 布谷鸟哈希（Cuckoo Hashing）：
  - 元素可以放在两个候选位置
  - 冲突时“踢走”已有元素
  - 存储的是完整元素



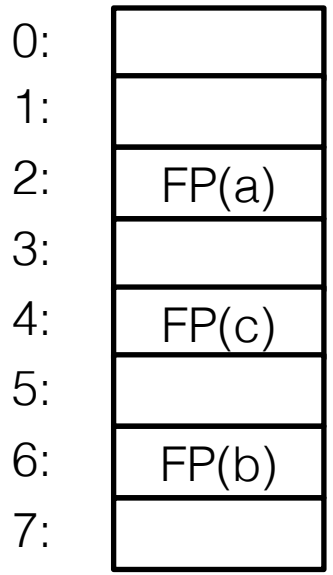
- **问题：**如果元素本身很长（例如基因序列），存储代价是不是太大？
- 思路：能否只存一个“简短的指纹”？

成员测试数据结构

数据结构	高性能	高空间利用率	删除支持
Bloom Filter	✓	✓	✗
Counting Bloom Filter	✓	✗	✓
Cuckoo Filter	✓	✓	✓

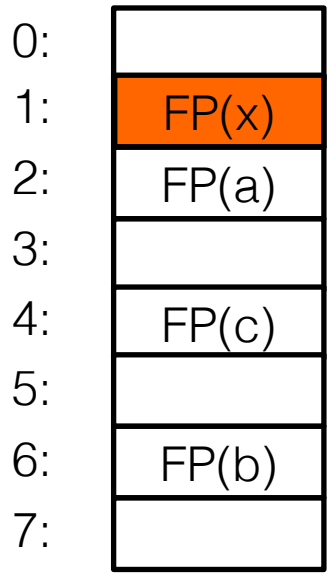
布谷鸟过滤器

- **思路：**将Fingerprint指纹存入哈希表
- **Fingerprint**(x): x的哈希值
  - 更长的指纹，更低的假阳性ε



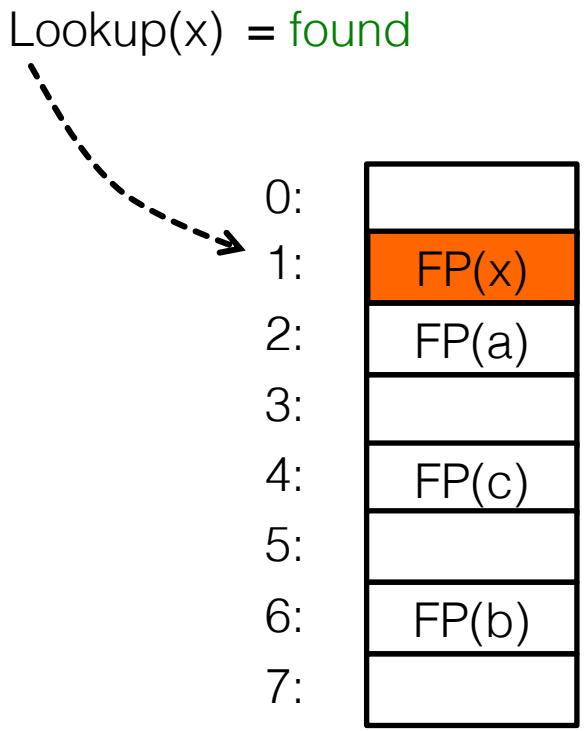
布谷鸟过滤器

- **思路：**将Fingerprint指纹存入哈希表
- **Fingerprint**(x): x的哈希值
  - 更长的指纹，更低的假阳性ε
- **Insert**(x):
  - 将 **Fingerprint(x)** 插入哈希表



布谷鸟过滤器

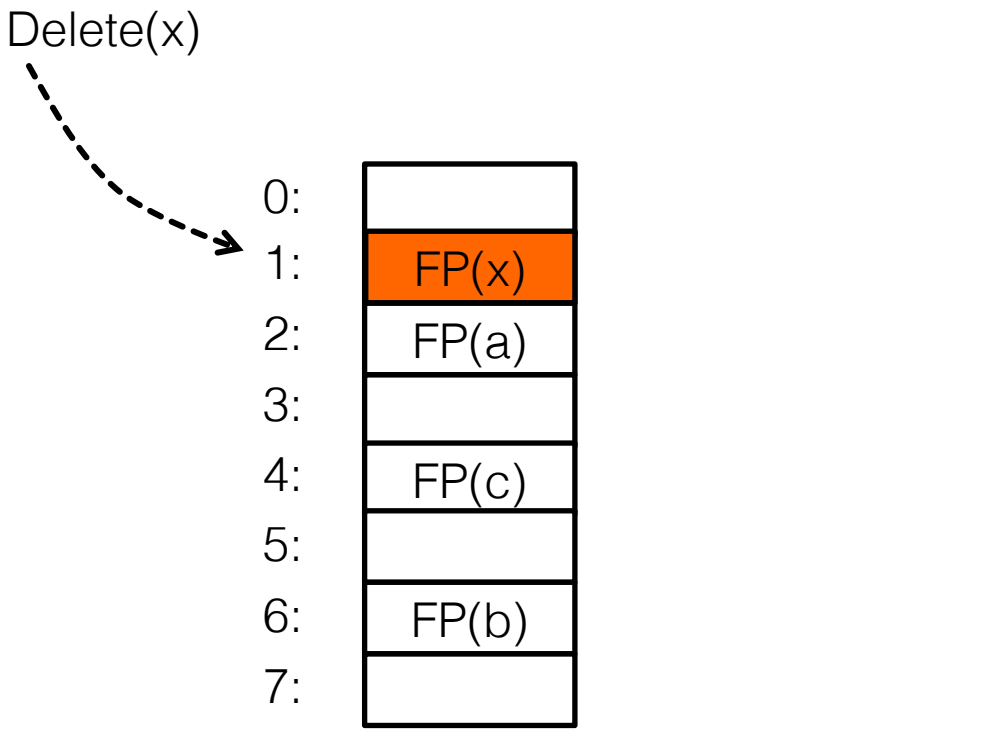
- **思路：**将Fingerprint指纹存入哈希表
- **Fingerprint**(x): x的哈希值
  - 更长的指纹，更低的假阳性ε
- **Insert**(x):
  - 将 **Fingerprint(x)** 插入哈希表
- **Lookup**(x):
  - 查找 **Fingerprint**(x)



布谷鸟过滤器

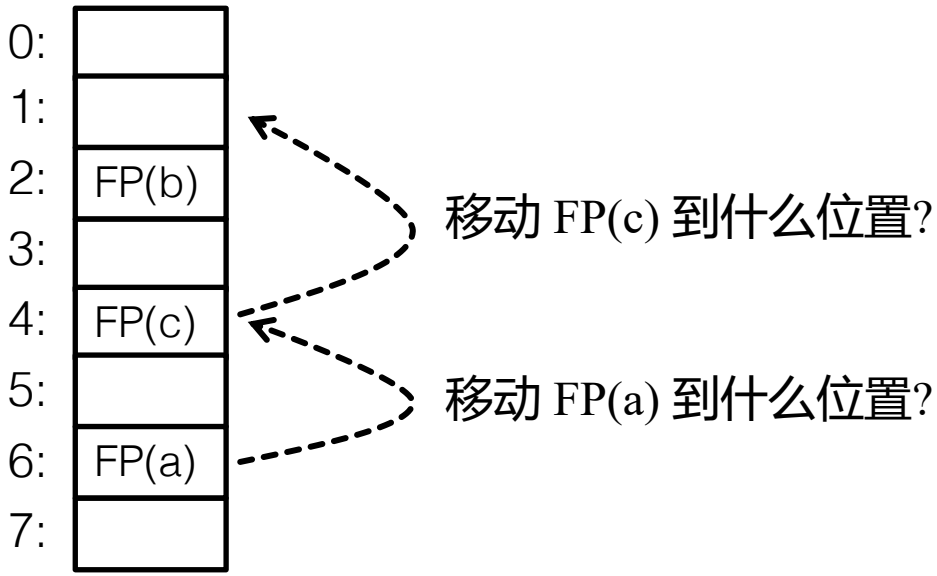
- **思路：**将Fingerprint指纹存入哈希表
- **Fingerprint**(x): x的哈希值（压缩）
  - 更长的指纹，更低的假阳性ε
- **Insert**(x):
  - 将 **Fingerprint(x)** 插入哈希表
- **Lookup**(x):
  - 查找 **Fingerprint**(x)
- **Delete**(x):
  - 删除 **Fingerprint**(x)

如何构造这样的数据结构?



布谷鸟哈希是否可行?

- 布谷鸟哈希需要重新哈希并移位已有的元素



在只有指纹的情况下
如何计算一个元素的移位的位置?

构造布谷鸟过滤器

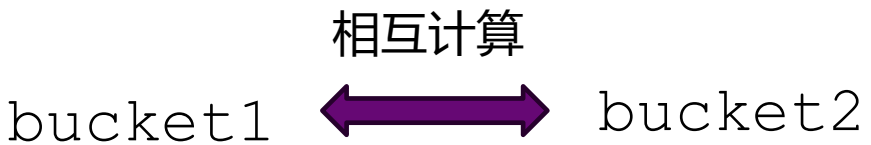
- 使用一个桶的信息和存储的指纹信息来推导另一个桶位置

$$bucket1 = hash(x)$$

$$bucket2 = bucket1 \oplus hash\big(FP(x)\big)$$

- 移位一个已有的指纹（元素）：

$$alternate(x) = current(x) \oplus hash\big(FP(x)\big)$$



好处

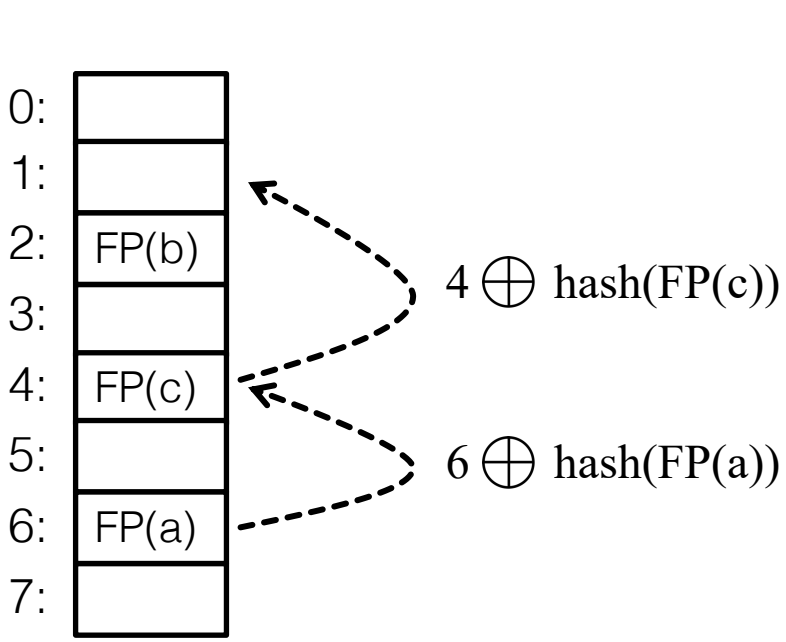
- **1. 只保存短指纹，而非完整 Key**
  - 降低空间开销，但失去通过原始 key 重新计算位置的能力
- **2. 两个候选桶可由 Fingerprint 相互推导**

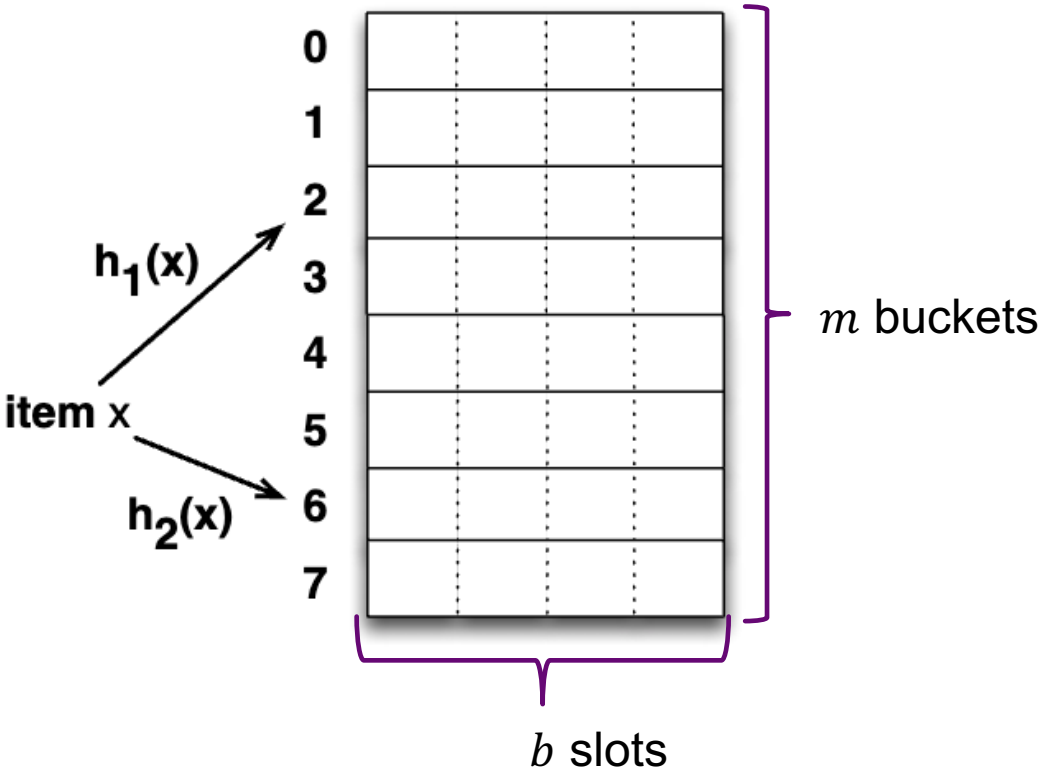
$$i_2 = i_1 \oplus h(f)$$

  - 因此元素被踢走后，只需 current bucket + fingerprint 即可找到另一个位置。
- **3. 支持删除已知插入的元素**
  - 可在两个候选桶中删除一个匹配 fingerprint；
  - **不能根据一次 positive query 随意删除，因为 positive 可能是 false positive。**

布谷鸟过滤器

- 重新定位已有的元素





算法

Algorithm 1: `Insert(x)`

f = fingerprint(x);
i_1 = hash(x);
$i_2 = i_1 \oplus$ hash(f);
if bucket[i_1] or bucket[i_2] has an empty entry **then**
    add f to that bucket;
    return `Done`;
// must relocate existing items;
i = randomly pick i_1 or i_2;
for $n = 0$; $n <$ MaxNumKicks; n++ **do**
    randomly select an entry e from bucket[i];
    swap f and the fingerprint stored in entry e;
    $i = i \oplus$ hash(f);
    if bucket[i] has an empty entry **then**
        add f to bucket[i];
        return `Done`;
// *Hashtable is considered full*;
return `Failure`;

Algorithm 2: `Lookup(x)`

f = fingerprint(x);
i_1 = hash(x);
$i_2 = i_1 \oplus$ hash(f);
if bucket[i_1] or bucket[i_2] has f **then**
    return `True`;
return `False`;

Algorithm 3: `Delete(x)`

f = fingerprint(x);
i_1 = hash(x);
$i_2 = i_1 \oplus$ hash(f);
if bucket[i_1] or bucket[i_2] has f **then**
    remove a copy of f from this bucket;
    return `True`;
return `False`;

概率分析

- 设 fingerprint 长度为 f bit，桶数为 m，并假设：$i_x = h(x), i_x' = i_x \oplus \Delta_x$，其中$\Delta_x = h\big(FP(x)\big)$。如果两个元素 fingerprint 相同：$FP(x) = FP(y)$，那么：$\Delta_x = \Delta_y = \Delta$，因此二者候选桶分别是：$\{i, i \oplus \Delta\}$和$\{j, j \oplus \Delta\}$。
- 两个不同元素产生相同 fingerprint，且候选桶集合相交（冲突）的概率：$\frac{1}{2^f} \cdot \frac{2}{m}$
  - 条件一：具有相同的指纹，其概率为$\frac{1}{2^f}$
  - 条件二：相同位置的概率为$\frac{2}{m}$
    - j 在 m 个桶上均匀
    - 使相交成立的 j 只有两个值（i 或 $i\ XOR\ \Delta$）
    - 概率 = $2/m$

插入失败

- 如果有 **2b+1 个元素**恰好映射到同一对桶 {u,v}，插入失败。

- **在随机哈希的模型下，期望出现“导致失败的两桶过载”事件的个数**。
- 将 n 个元素插入到一个空的大小为 $m = cn$ 的表中, c是一个常数，且每个桶的大小为b。每当有 $q = 2b + 1$个元素映射到同两个桶时，插入操作就会失败，平均期望为

$$\binom{n}{2b+1}\left(\frac{2}{2^f \cdot m}\right)^{2b} = \binom{n}{2b+1}\left(\frac{2}{2^f \cdot cn}\right)^{2b} = \Omega\left(\frac{n}{4^{bf}}\right) \Rightarrow f = \Omega(\log n\,/b)$$

- Bucket 越大 b ↑，允许的 fingerprint 越短；集合规模越大 n ↑，为了控制这种碰撞导致的失败，需要更长的 fingerprint。

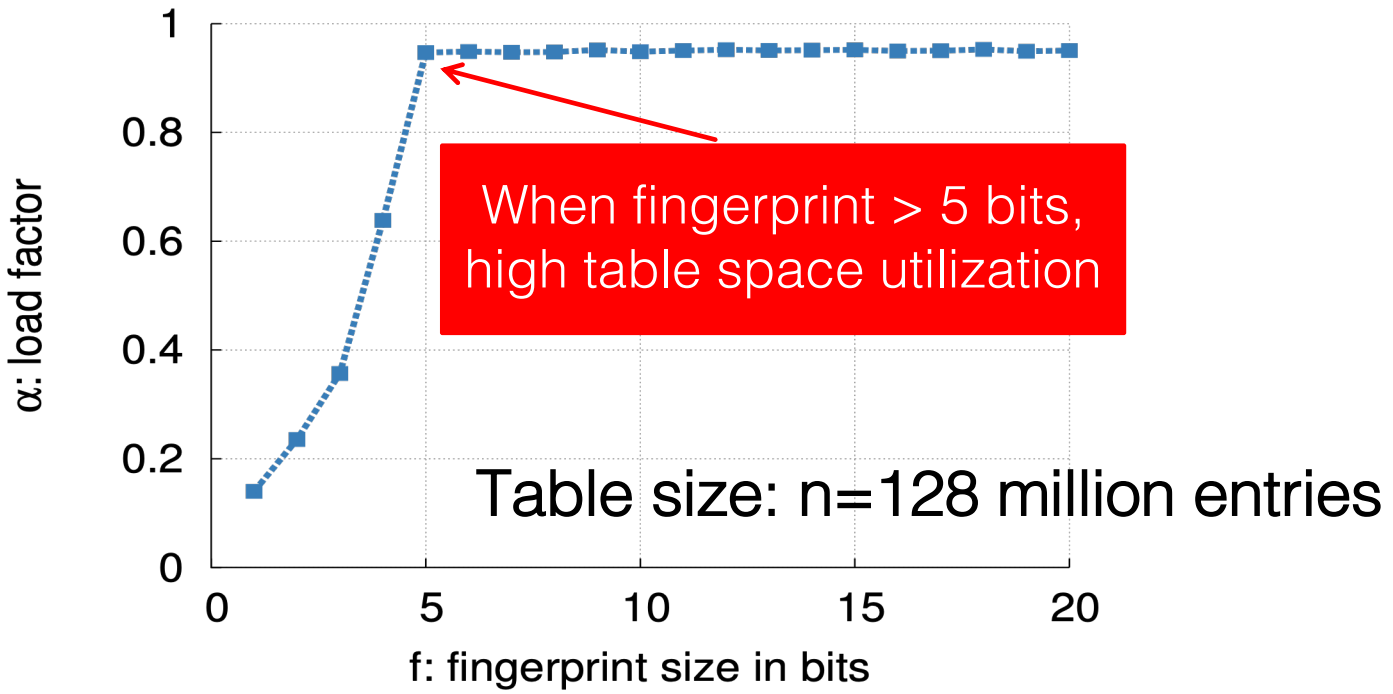
指纹大小设置

- 查询与一个存储的指纹匹配并返回一个假阳性成功匹配的概率最多为$1/2^f$。
- 在进行$2b$次比较后，假指纹命中的总概率的上界为：$1-\left(1-\frac{1}{2^f}\right)^{2b} \approx \frac{2b}{2^f} \leq \epsilon$ (假阳性) $\Rightarrow f \geq \lceil \log_2(2b/\epsilon) \rceil$

- **单位**空间复杂度：$C = \frac{\text{table size}}{\#\text{items}} = \frac{f \cdot (\#\text{entries})}{\alpha \cdot (\#\text{entries})} = \frac{f}{\alpha}$ bits
  - entries是表里所有slots，#items是**当前实际插入并存放在 Cuckoo Filter 里的元素数量**
  - 装载因子 α：$\alpha = \frac{\#\text{items}}{\#\text{entries}}$
  - f 越大 → 指纹更长，误判率更低，但空间开销更大。
  - α越大 → 装得更满，利用率高，平均开销更小。

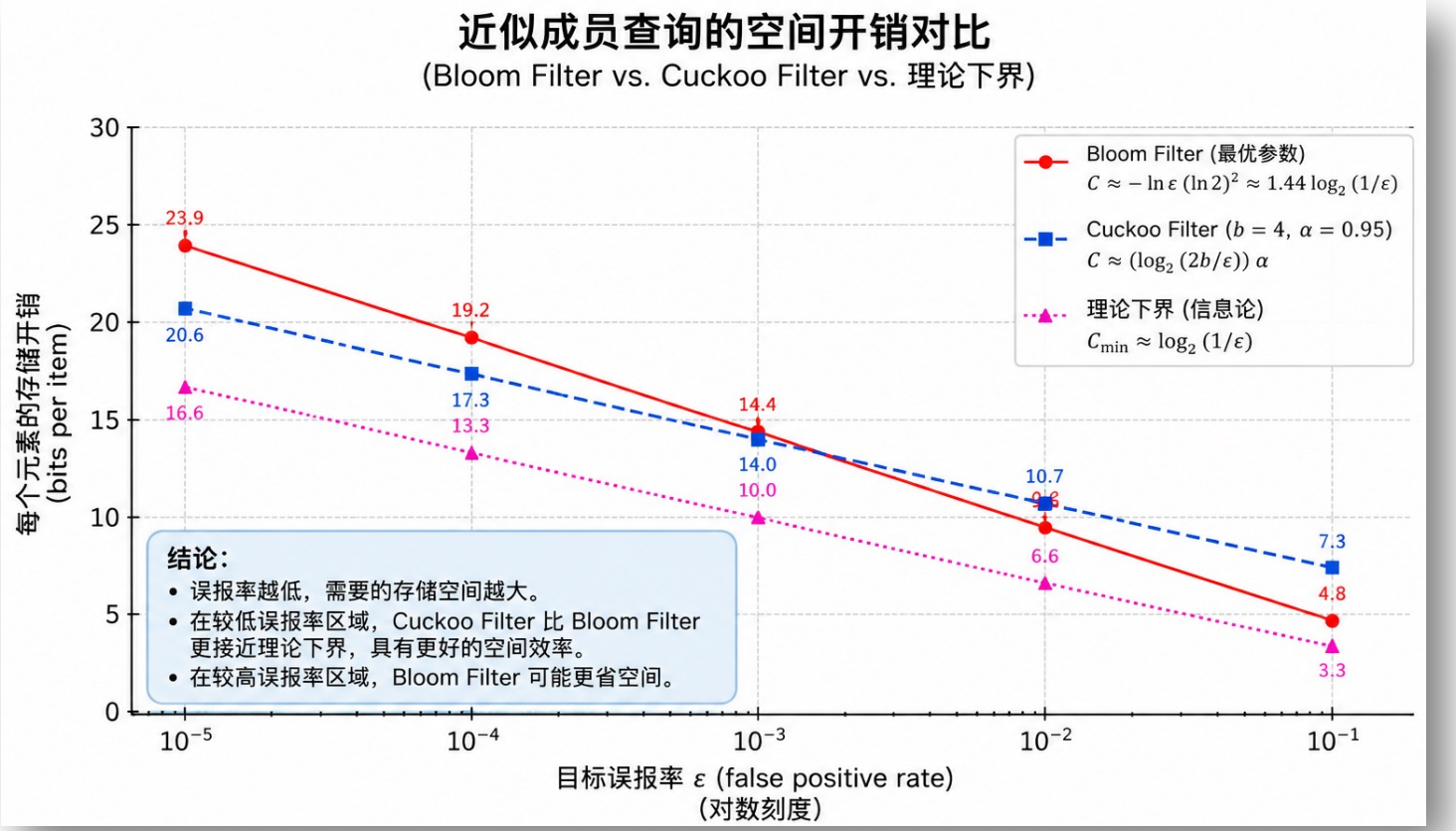
指纹大小设置

- Fingerprint 长度对 Cuckoo Filter 装载率的影响
  - 当 fingerprint 过短时，相同 fingerprint 与候选桶对的重复会导致局部过载，使插入提前失败
  - 随着f增大，装载率迅速提升并趋近 Cuckoo 结构本身的上限。

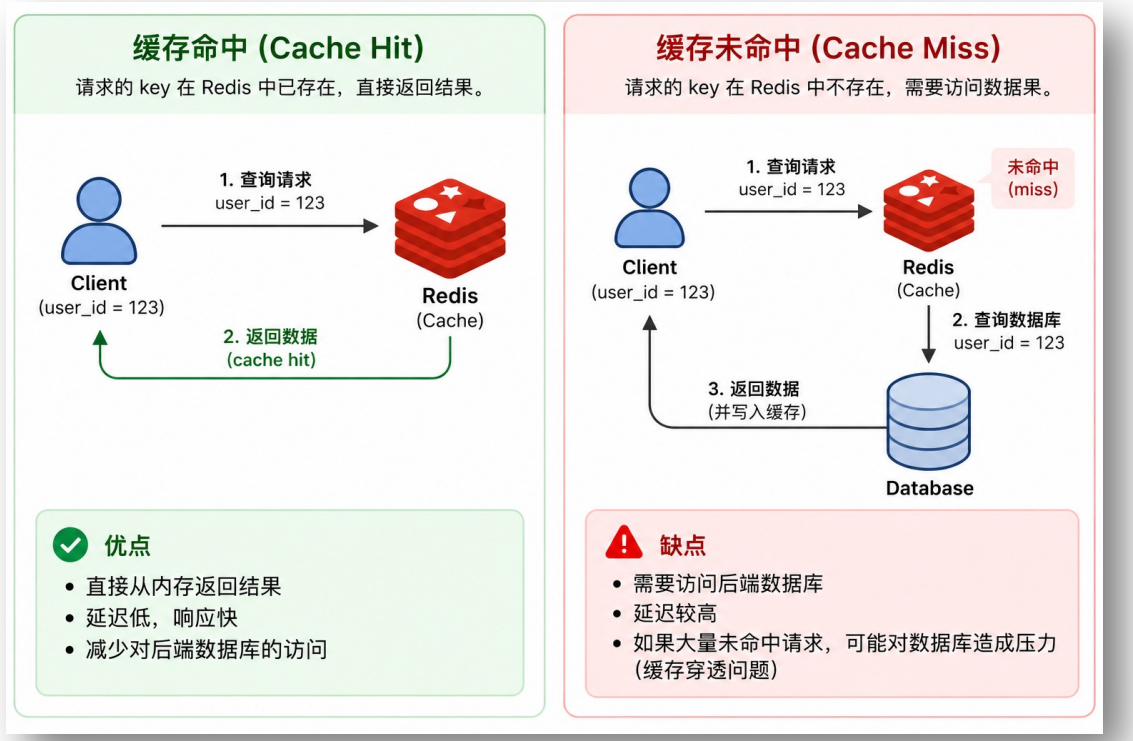


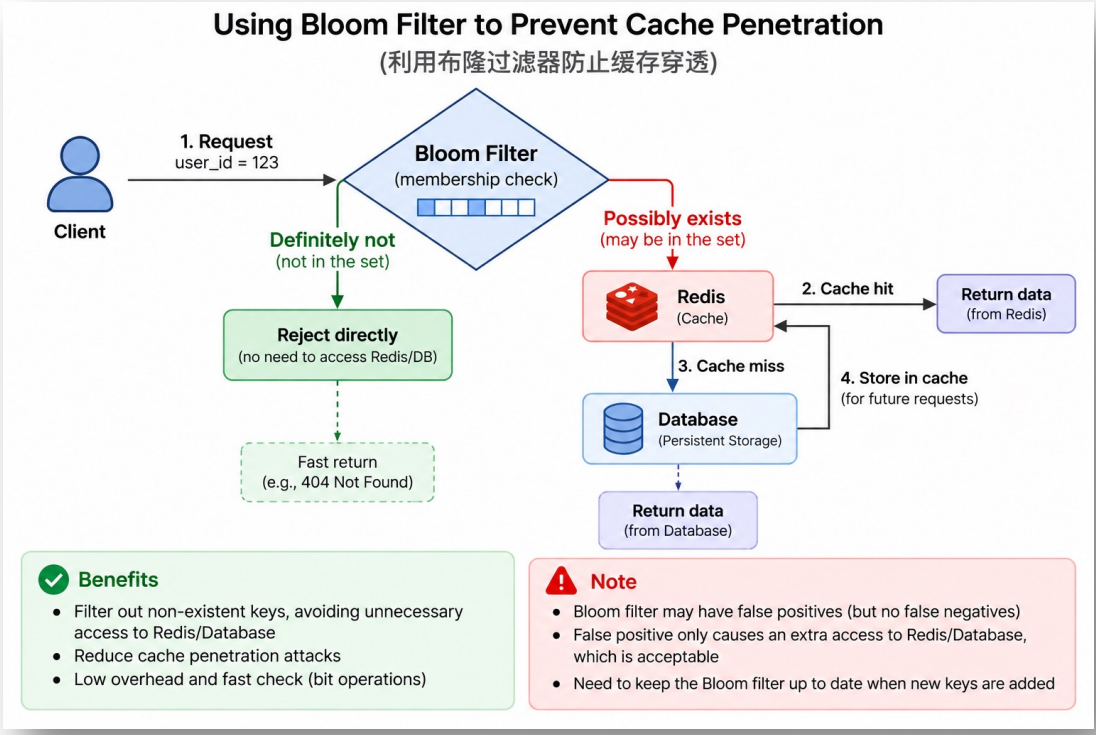
空间复杂度

- Space–accuracy trade-off：误报率越低，需要的 bits/item 越多；在低误报率区域，Cuckoo Filter 比 Bloom Filter 更接近理论空间下界。



布隆/布谷鸟过滤器应用：防止 **Cache Penetration**





Redis

- Redis（全称Remote Dictionary Server ）是一个开源的、基于内存的高性能键值对（Key-Value）非关系型数据库
- **核心特点**
  - **内存存储**：数据主要保存在内存中，读写速度极快（通常操作小于 1 毫秒）
  - **丰富的数据结构**：不仅支持简单的字符串（String），还支持哈希（Hash）、列表（List）、集合（Set）和有序集合（Sorted Set）等
  - **支持持久化**：可以将内存中的数据保存到硬盘中，防止断电或重启导致数据丢失
  - **高并发**：支持高并发的读写请求
- **常见应用场景**
  - **缓存**：把经常访问的热点数据放在 Redis 中，减轻主数据库（如 MySQL）的压力
  - **会话存储**：用来保存用户的登录状态（Session）
  - **消息队列**：利用其发布/订阅或列表特性来实现简单的消息队列
  - **计数器与排行榜**：利用原子计数和有序集合快速实现点赞数、热搜榜等功



流式统计问题：基数统计

- **前面我们已经解决了“是否出现过”**。现在如果问题变成：**“有多少个不同用户访问过？”**
- **基数统计**
  - 基数 (Cardinality) 是指一个字段所包含的不同取值的个数，有时候也称为 Distinct Values，简写为 DV。
- 举个例子：
  - 序列 [1, 2, 3, 4] 的基数为 4，因为包含 4 个不同的取值。
  - 序列 [1, 2, 3, 1, 2] 的基数为 3，虽然包含 5 个元素，但其中的 1, 2 分别重复了一次。
- 真实场景
  - 网站 UV（Unique Visitors）：今天来了多少个不同用户？
  - 网络监控：出现过多少个不同 IP？
  - 搜索：有多少个不同 query？
  - 数据库：某字段有多少 distinct values？

HashSet

- 最直观的基数统计方法是利用 HashSet：将序列中的所有值依次添加到 HashSet 中，最后统计 HashSet 中值的个数即可。

```
def get_dv(stream):
    s = set()
    for value in stream:
        s.add(value)
    return len(s)
```

- 效率低的原因：空间成本。当要统计的数据非常多时，HashSet 将会占用很大的内存。
- 如果有 10 亿个不同 user ID：即使每个只占 8 bytes：$8\times10^9 \approx 8GB$

从概率视角看计数方法

- 精确的记录需要记录已经出现过的不同元素，空间随集合规模增长。
- 概率论提供了另一种思路：**通过观察“有多罕见的事件曾经发生过”，反推大约进行了多少次独立试验。**
- 若某事件每次发生概率为 p，则首次观察到它所需的试验次数期望为：$\boxed{\mathbb{E}[T] = 1/p}$
- 因而，若观察到了概率约为 2^{-r}的稀有事件，就暗示试验次数大约处在：$\boxed{N \sim 2^r}$
  - 例子：在摇骰子猜大小的游戏中，三个骰子同时为 6 点的概率很小，为 1/(6^3)。假如在某场游戏中摇出了三个 6 点，猜猜一共摇了几次？

HyperLogLog 算法的原理

- 对于每一条待统计的数据（例如 user_id），计算其 hash 值并写成二进制形式（0-1 串），然后将其看作一局抛硬币游戏的记录。其中：
  - 0 代表硬币正面朝上。
  - 1 代表硬币反面朝上。

- 如 hash(uid_345678)=**0001**0010，意味着这局抛硬币游戏出现连续 3 次正面朝上，第 4 次反面朝上，游戏结束，最终得分为 4。第 4 位以后的序列不影响得分，可以忽略。

- 按照上述步骤，计算每条数据对应的「得分」，并找出其中的最高分 μ。那么这组数据的基数的粗略估计可以为：$N = 2^{\mu}$

HyperLogLog 算法的原理

- 在上述过程中有一个重要步骤：首先对每个待观察的数据进行 Hash。为什么不能直接观察数据本身对应的二进制串呢？
- 因为 HLL 所依赖的概率模型要求观察到的比特串近似均匀随机。对于理想的 Hash 输出，每一位出现 0 或 1 的概率均约为$1/2$，因此连续出现 k 个 0 的概率约为 2^{-k}。**连续 0 越多，这一事件就越罕见，而观察到这样的罕见事件通常意味着已经进行了大量独立观察。**
- 如果直接使用原始数据的二进制表示，则无法保证这种均匀性。例如，对于较小的整数 1、2、3，其固定长度二进制表示中天然包含大量前导 0。如果直接统计这些前导 0，就会把“数值较小”错误地解释成“观察到了一个极小概率事件”，从而导致严重的估计偏差。

HyperLogLog 算法的原理

- 因此，HLL 首先利用 Hash 将任意输入映射为近似均匀分布的比特串，再根据其中连续 0 的长度进行基数估计。HyperLogLog 算法本身并不限定具体的 Hash 函数，实际实现通常选择速度快且输出分布均匀的 Hash 函数。例如，Redis 的 HyperLogLog 使用 **64-bit MurmurHash2**。
- 类似的概率思想也出现在 Bitcoin 的 Proof of Work 中。Bitcoin 要求区块头的 Hash 值小于给定的 target。target 越小，满足条件的 Hash 越罕见，因此平均需要尝试的 Hash 次数越多。用“要求更多前导 0”可以直观理解这一过程。
- 两者背后的共同思想都是：

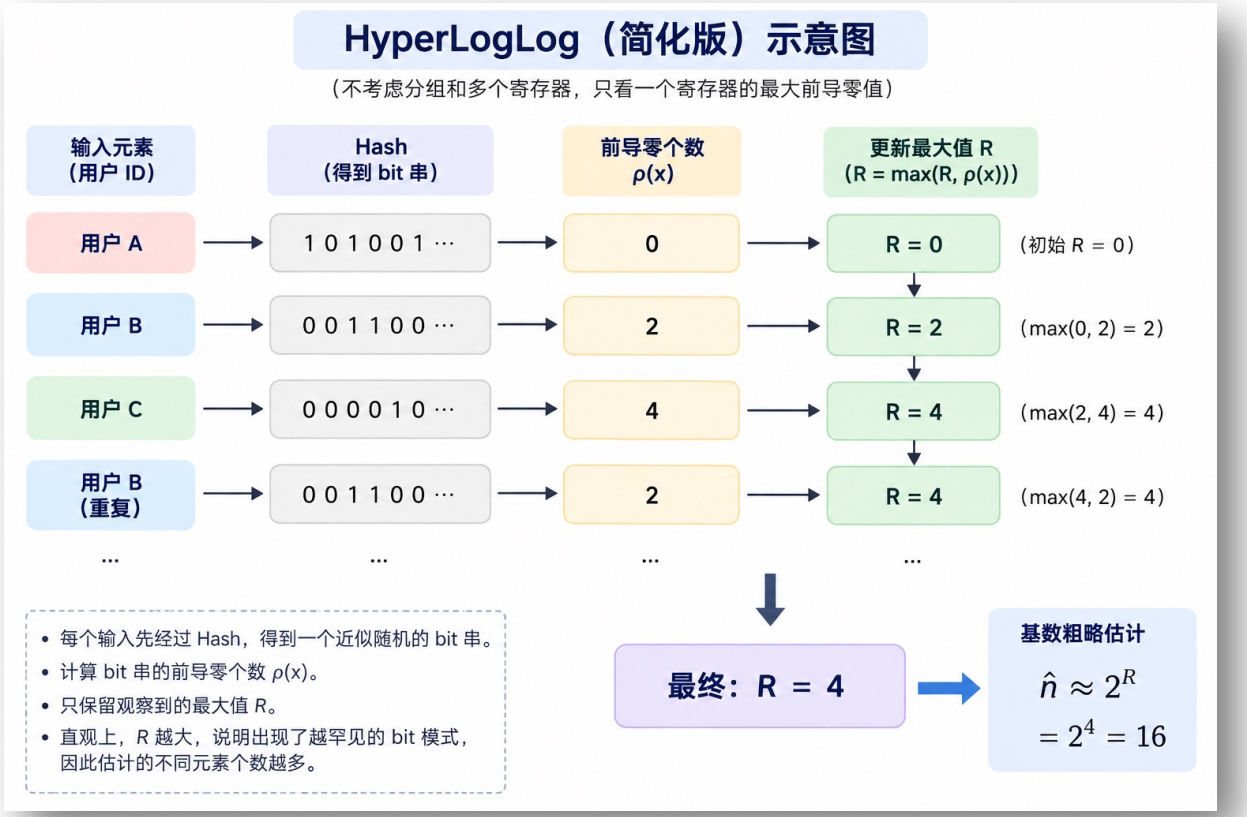
$$\boxed{\text{越罕见的 Hash 结果} \Rightarrow \text{通常意味着需要经历更多次独立尝试}}$$

- 区别在于，HLL 利用这一现象估计数据的基数，而 Bitcoin 利用这一现象衡量计算工作量。

基数估计算法

```
def get_dv(stream):
    max_z = 0
    for value in stream:
        h = murmur_hash(value)
        z = leading_zeros_count(h)
        if z > max_z:
            max_z = z
    return 1 << (max_z + 1)
```

- 在数据量较小的情况下误差非常大，因为前导零个数的偶然性太大了
- 最关键的是，这些偏差无法靠算法本身来控制，准确度全靠运气



算法优化

- 一种最简单的思路就是「多轮分拆计算求平均值」：把输入数据均分为 m 份（称为桶），每一个桶分别应用基数估计算法，最终得分 $\bar{\mu}$ **为各桶得分的平均值**
- 算法的计算公式可表示为：

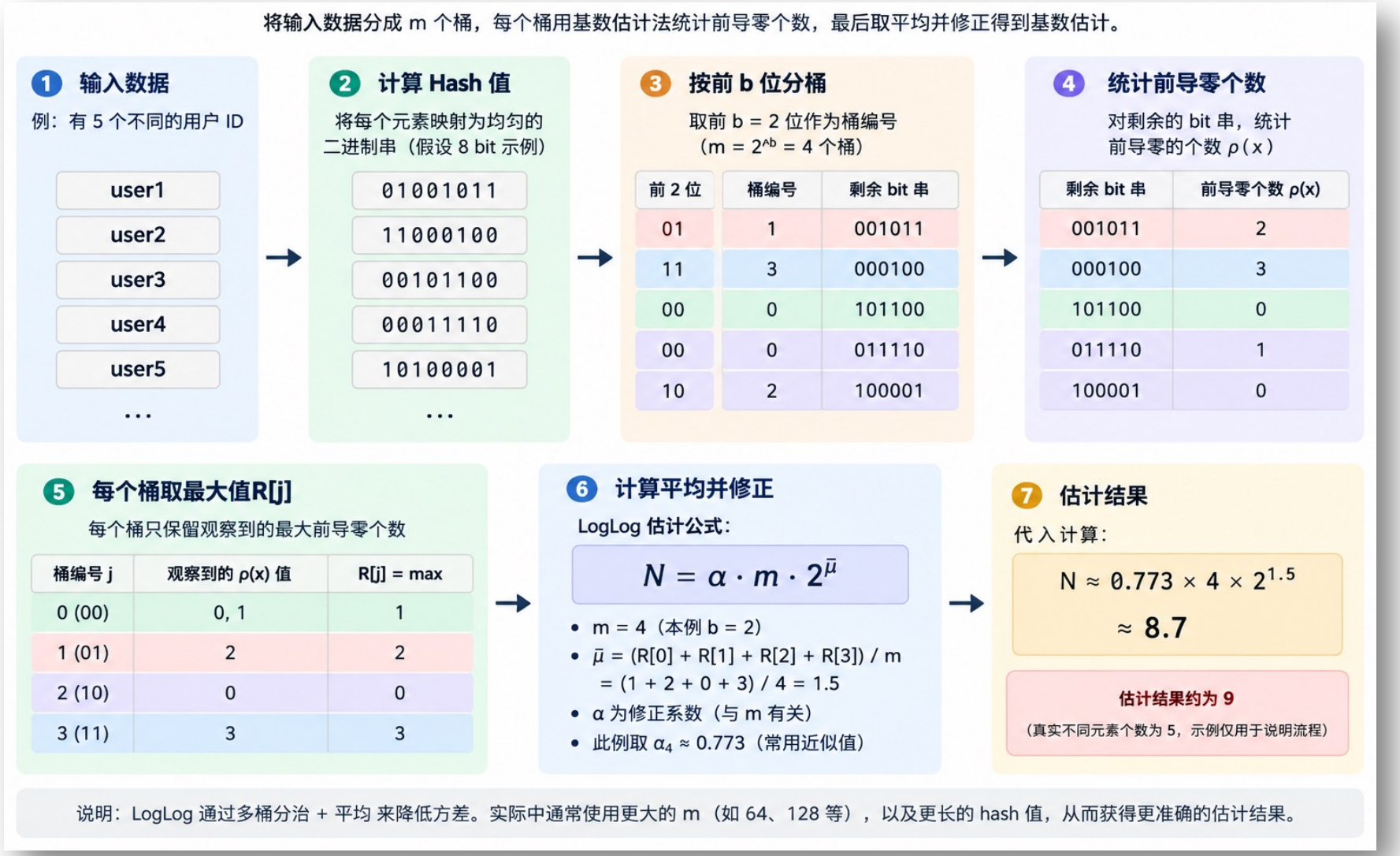
$$N = \alpha \cdot m \cdot 2^{\bar{\mu}}$$

- 其中，m 为分桶个数，$\bar{\mu}$为各桶最高得分的平均值，α 为修正系数，用于修正算术平均数带来的系统偏差。α 的计算规则如下：

$$\alpha_m = \left(m \int_0^\infty \left(log_2 \frac{2+u}{1+u} \right)^m \mathrm{d}u \right)^{-1} \approx \begin{cases} 0.673 & m = 16 \\ 0.697 & m = 32 \\ 0.709 & m = 64 \\ \frac{0.7213}{1+1.079/m} & m \geq 128 \end{cases}$$

- 根据算法的特点，通常将分桶数 m 设为 2 的整数次幂。例如 $m = 64 = 2^6$，此时**可以通过 hash 值的前 6 个 bit 来表示桶编号。从第 7 个 bit 开始统计前导零个数。**

例子


将输入数据分成 m 个桶，每个桶用基数估计法统计前导零个数，最后取平均并修正得到基数估计。
1 输入数据
例：有 5 个不同的用户 ID
user1
user2
user3
user4
user5
...
2 计算 Hash 值
将每个元素映射为均匀的二进制串（假设 8 bit 示例）
01001011
11000100
00101100
00011110
10100001
...
3 按前 b 位分桶
取前 b = 2 位作为桶编号
（m = 2^b = 4 个桶）
前 2 位 | 桶编号 | 剩余 bit 串
01 | 1 | 001011
11 | 3 | 000100
00 | 0 | 101100
00 | 0 | 011110
10 | 2 | 100001
4 统计前导零个数
对剩余的 bit 串，统计前导零的个数 ρ(x)
剩余 bit 串 | 前导零个数 ρ(x)
001011 | 2
000100 | 3
101100 | 0
011110 | 1
100001 | 0
5 每个桶取最大值R[j]
每个桶只保留观察到的最大前导零个数
桶编号 j | 观察到的 ρ(x) 值 | R[j] = max
0 (00) | 0, 1 | 1
1 (01) | 2 | 2
2 (10) | 0 | 0
3 (11) | 3 | 3
6 计算平均并修正
LogLog 估计公式：
$N = \alpha \cdot m \cdot 2^{\bar{\mu}}$
m = 4（本例 b = 2）
$\bar{\mu}$ = (R[0] + R[1] + R[2] + R[3]) / m = (1 + 2 + 0 + 3) / 4 = 1.5
α 为修正系数（与 m 有关）
此例取 $\alpha_4 \approx 0.773$（常用近似值）
7 估计结果
代入计算：
$N \approx 0.773 \times 4 \times 2^{1.5}$
≈ 8.7
估计结果约为 9
（真实不同元素个数为 5，示例仅用于说明流程）
说明：LogLog 通过多桶分治 + 平均 来降低方差。实际中通常使用更大的 m（如 64、128 等），以及更长的 hash 值，从而获得更准确的估计结果。

算法优化

- 算法通过「分桶求平均值」的方式提高了估算结果的稳定性，使得算法更能抵御偶然性带来的影响。
- 但这么做仍然不够，因为算术平均数有一个天然的缺陷：受到极大/极小值的影响
  - 一个离群点可能把最终结果严重带偏。例如老板月入 100000 元，9 个员工均月入 3000 元，那么平均收入就是 (100000+3000*9)/10=12700 元，这距离群里中的大多数（即 9 个员工）相差甚远，不能反映普遍情况
  - 如果将算术平均数改为**调和平均数**就可以解决这个问题，调和平均数的计算公式如下：

$$\bar{x} = \frac{1}{\frac{1}{n}\sum_n \frac{1}{x_i}} = \frac{n}{\sum_n \frac{1}{x_i}}$$

  - 使用调和平均数计算出的平均收入为 10/(1/100000+9/3000)=3322，比较接近群体中的普遍情况。
- HyperLogLog 算法对于算法的重要改进就是把算术平均数改成了调和平均数。公式如下：

$$N = \alpha \cdot m \cdot \frac{m}{\sum_{i=1}^{m} \frac{1}{2^{\mu_i}}}$$

HyperLogLog（HLL）算法

- 在实际使用中，为了**提高小样本的准确度**，HLL 在计算结果的基础上还进行了一次修正。
- 更多桶 ⇒ 更多近似独立的统计观测 ⇒ 偶然波动相互抵消 ⇒ 误差按 $1/\sqrt{m}$ 下降

Let $h : \mathcal{D} \to \{0,1\}^{32}$ *hash data from* $\mathcal{D}$ *to binary 32–bit words.*
Let $\rho(s)$ *be the position of the leftmost 1-bit of* s*: e.g.,* $\rho(1\cdots) = 1$, $\rho(0001\cdots) = 4$, $\rho(0^K) = K+1$.
define $\alpha_{16} = 0.673$; $\alpha_{32} = 0.697$; $\alpha_{64} = 0.709$; $\alpha_m = 0.7213/(1+1.079/m)$ for $m \geq 128$;
Program HYPERLOGLOG (**input** $\mathcal{M}$: multiset of items from domain $\mathcal{D}$).
assume $m = 2^b$ with $b \in [4\,..\,16]$.
initialize a collection of m registers, $M[1], \ldots, M[m]$, to 0;

for $v \in \mathcal{M}$ **do**
  set $x := h(v)$;
  set $j = 1 + \langle x_1 x_2 \cdots x_b \rangle_2$; {*the binary address determined by the first* b *bits of* x}
  set $w := x_{b+1} x_{b+2} \cdots$;
  set $M[j] := \max(M[j], \rho(w))$;
compute $E := \alpha_m m^2 \cdot \left(\sum_{j=1}^{m} 2^{-M[j]} \right)^{-1}$; {*the "raw" HyperLogLog estimate*}
if $E \leq \frac{5}{2}m$ **then**
  let V be the number of registers equal to 0;
  if $V \neq 0$ **then set** $E^\star := m \log(m/V)$ **else set** $E^\star := E$; {*small range correction*}
if $E \leq \frac{1}{30} 2^{32}$ **then**
  set $E^\star := E$; {*intermediate range—no correction*}
if $E > \frac{1}{30} 2^{32}$ **then**
  set $E^\star := -2^{32} \log(1 - E/2^{32})$; {*large range correction*}
return *cardinality estimate* $E^\star$ *with typical relative error* $\pm 1.04/\sqrt{m}$.

小基数	中等基数	大基数
问题： 大量寄存器仍为空，HLL 的前导零统计还不稳定	问题： HLL 进入稳定工作区间	问题： 32-bit Hash 空间开始发生明显碰撞
利用信息： 空寄存器数量 V	利用信息： 各桶最大前导零 M_j	利用信息： Hash 空间占用率
$E^* = m \ln \frac{m}{V}$	$E^* = E$	$E^* = -2^{32} \ln\left(1 - \frac{E}{2^{32}}\right)$
条件：$E \leq 2.5m, V \neq 0$	$2.5m < E \leq 2^{32}/30$	$E > 2^{32}/30$
本质：Linear Counting	直接使用 HLL	修正 Hash collision

Linear Counting

- **还有多少个寄存器从来没有被任何元素访问过？**
  - m：寄存器总数量。
  - V：空寄存器数量。
  - n：实际不同元素数量。
- **假设每个不同元素经过 Hash 后，均匀随机地映射到一个寄存器。**
- **第一步：一个元素没有落到某个寄存器的概率**
  - 因为有m个寄存器：$P(\text{没有落入}) = 1 - \frac{1}{m}$
- **第二步：插入n个不同元素后，该寄存器仍为空的概率**
  - $P(\text{仍为空}) = \left(1 - \frac{1}{m}\right)^n$，当$m$较大时：$\left(1 - \frac{1}{m}\right)^n \approx e^{-n/m}$
- **第三步：用空寄存器比例估计这个概率**
  - 因为总共有m个寄存器，其中V个为空：$\frac{V}{m} \approx e^{-n/m}$
  - 两边取自然对数：$\ln\frac{V}{m} \approx -\frac{n}{m}$，整理：$\boxed{E^* = m\ln\frac{m}{V}}$

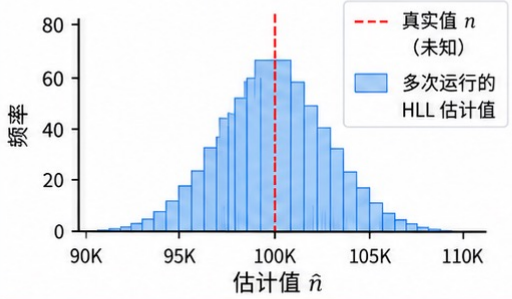
如何利用 Hash 空间占用率进行修正?

- $N = 2^{32}$：Hash 空间大小；n：实际不同元素数量（未知）；E：HLL 原始估计值；E^*：修正后的估计值。
- **第一步：某个 Hash 值没有被选中的概率**
  - 对于任意一个固定的 Hash 值：$P(\text{一次未选中}) = 1 - \frac{1}{N}$
  - 假设有 n个不同 UID，各自独立均匀地产生 Hash 值，那么，这个 Hash 值始终未被使用的概率是：$P(\text{未使用}) = \left(1 - \frac{1}{N}\right)^n$，当$N$很大时：$\boxed{P(\text{未使用}) \approx e^{-n/N}}$
- **第二步：计算被使用的 Hash 值数量**
  - 一个 Hash 值至少被使用一次的概率是：$1 - e^{-n/N}$
  - 因为总共有 N个可能的 Hash 值，所以不同 Hash 值数量的期望约为：$E = N\left(1 - e^{-n/N}\right)$
- **第三步：反推出实际数量 \(n\)**
  - 由：$E = N\left(1 - e^{-n/N}\right)$，得到：$\frac{E}{N} = 1 - e^{-n/N}$
  - 因此：$e^{-n/N} = 1 - \frac{E}{N}$，两边取自然对数：$-\frac{n}{N} = \ln\left(1 - \frac{E}{N}\right)$
  - 整理：$\boxed{n = -N \ln\left(1 - \frac{E}{N}\right)}$，代入：\[N=2^{32} \]
  - 最终得到：$\boxed{E^* = -2^{32} \ln\left(1 - \frac{E}{2^{32}}\right)}$

HyperLogLog的估计与误差分析

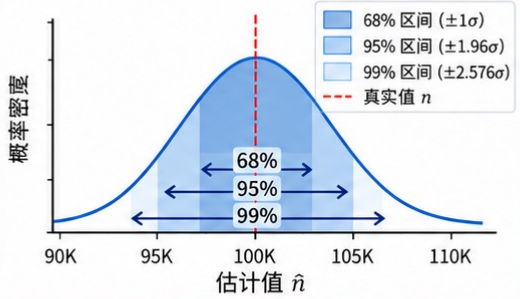
1. 估计值的分布（多次实验）

对同一真实基数 n，使用不同随机哈希进行多次实验，HLL 的估计值呈近似正态分布。



2. 近似正态分布与置信区间

在理想条件下，HLL 估计值可近似为正态分布，置信区间由相对标准误差（RSE）给出。



3. 数值示例（$m = 1024$）

寄存器数量：$m = 1024$

相对标准误差（RSE）：$r = \frac{1.04}{\sqrt{m}} = \frac{1.04}{32} = 3.25\%$

95% 临界值：$z_{0.975} = 1.96$

95% 相对误差：$1.96 \times 3.25\% = 6.37\%$

若估计值 $\hat{n} = 100000$，则 95% 置信区间为：

$$\left[\frac{\hat{n}}{1+zr}, \frac{\hat{n}}{1-zr}\right] = [94{,}011\ ,\ 106{,}803]$$

（其中 $r = 1.04/\sqrt{m}$，$z = 1.96$）

4. 置信区间的计算方法

1. HLL 的相对标准误差（RSE）：
$$r = \frac{1.04}{\sqrt{m}}$$
2. 在置信水平 $1-\alpha$ 下（近似正态）：
$$P\left(|\hat{n} - n| \le z_{\alpha/2} \cdot r \cdot n\right) \approx 1 - \alpha$$
3. 由此得到真实值的置信区间：
$$\frac{\hat{n}}{1+z_{\alpha/2}r} \le n \le \frac{\hat{n}}{1-z_{\alpha/2}r}$$

常用的 z 值：
68%：$z = 1.00$　95%：$z = 1.96$　99%：$z = 2.576$

5. 不同寄存器数量 m 的误差水平

寄存器数量 m	相对标准误差 RSE $= 1.04/\sqrt{m}$	约 95% 误差范围 ($\pm 1.96 \times$ RSE)
16	26.00%	±51.0%
64	13.00%	±25.5%
256	6.50%	±12.7%
1024	**3.25%**	**±6.37%**
4096	1.63%	±3.19%
16384	0.81%	±1.59%

规律： $m \times 4 \Rightarrow RSE \times \frac{1}{2}$

（要让误差降低一半，需要约 4 倍的寄存器空间）

6. 关键结论

1. HLL 的误差用相对标准误差（RSE）衡量：RSE $= 1.04/\sqrt{m}$。
2. RSE 描述的是估计值相对于真实基数的随机波动大小（在适当条件下近似正态），而不是用来修正估计偏差的系统性偏差。
3. 可以给出某个置信水平下的误差区间，通常基于正态近似计算。
4. 不能知道这一次估计的真实误差，只能给出统计意义上的误差范围。
5. 当 n 很小时需要使用 Linear Counting，当 n 很大时需要修正 Hash 碰撞的影响，或者采用 HLL++ 进行进一步的偏差校正。

一句话总结： HyperLogLog 不能给出精确误差，但可以量化估计的典型误差和置信区间（以 RSE $= 1.04/\sqrt{m}$ 为核心）。

相对标准差的定义：

$$RSE = \frac{\sqrt{\mathrm{Var}(\hat{n})}}{n}$$

假设 HLL 最终算出来

$$\hat{N} = 10000$$

并且 $m = 1024$，那么

$$\mathrm{RSD} \approx \frac{1.04}{\sqrt{1024}} = \frac{1.04}{32} \approx 3.25\%$$

估计值的标准差大约为

$$10000 \times 3.25\% = 325$$

所以可以理解成：

$$\hat{N} = 10000 \pm 325$$

HyperLogLog的流式计算

1. 数据流输入（Streaming）

数据以流的形式逐个到来：

x_1 = "apple"

x_2 = "banana"

x_3 = "apple"

x_4 = "orange"

x_5 = "cat"

……

2. 单个元素的更新过程（O(1)）

（1）Hash

$h(x)$

110010…0110

前 p 位 选择桶

后续位 计算前导零

（2）选择桶

桶编号 j = 前 p 位

$j = 6$

（例如 $p = 4$，共有 $m = 16$ 个桶）

（3）计算前导零

$\rho(x)$ = 后续位的前导零数量

$\rho(x) = 3$

（4）更新寄存器

$M_j \leftarrow \max(M_j, \rho(x))$

原值：$M_6 = 2$

新值：$\max(2, 3) = 3$

3. 维护紧凑的摘要

一个长度为 m 的寄存器数组（每个桶记录最大前导零数）

桶 0	1
桶 1	4
桶 2	0
桶 3	2
桶 4	1
桶 5	3
桶 6	3
⋮	⋮
桶 $m-1$	0

4. 需要结果时再估计（Query）

遍历所有 m 个桶，根据寄存器值计算估计结果：

$$E = \alpha_m \frac{m^2}{\sum_{j=1}^{m} 2^{-M_j}}$$

- 时间复杂度：$O(m)$
- 可加入小基数/大基数修正
- 不需要重新处理原始数据

5. HyperLogLog 的流式特性

1. **一次处理，不保存原始数据**
   每个元素到来时立即更新，只维护 m 个桶的摘要。
2. **内存占用固定且很小**
   空间复杂度 $O(m)$，与数据规模 n 无关。
3. **支持在线处理和实时估计**
   可以随时输出当前的估计值（如果维护一些额外计数）。
4. **可合并（Mergeable）**
   两个 HLL 摘要可以按桶取 max 合并，便于分布式计算。

6. 复杂度总结

操作	时间复杂度
处理一个元素 (Update)	$O(1)$
处理 n 个元素	$O(n)$
估计基数 (Query)	$O(m)$
空间复杂度	$O(m)$

总结：HyperLogLog 是一个典型的流式算法，
用固定的内存、单次 O(1) 的更新，支持大规模数据的实时去重估计。

HyperLogLog的应用

- **1. Apache DataSketch**
  - 算法族中包含 HyperLogLog 的实现，被广泛用于许多大数据基础组件中，用于支持基数、分位数等的快速计算。例如：Hive/Spark， PostgreSQL、
- **2. Redis**
  - Redis 中使用 PFCOUNT 命令来调用 HLL 算法。
  - 其 HLL 结构使用了$2^{14} = 16384$个桶，hash 值采用 64bit 表示，除了桶编号之外剩余的 50 bit (64-14=50) 全部用于统计得分。为了确保桶中记录的分数最大范围高于 50，每个桶需要占用 6 bit 空间（ $2^6 > 50$ ）。这样，总体的空间占用为 $16384 * 6\ bit = 12KB$。



Apache DataSketches 是一个开源的分布式、高性能、大规模流式资源库，专门用于**在海量数据中进行快速、近似的统计分析**。

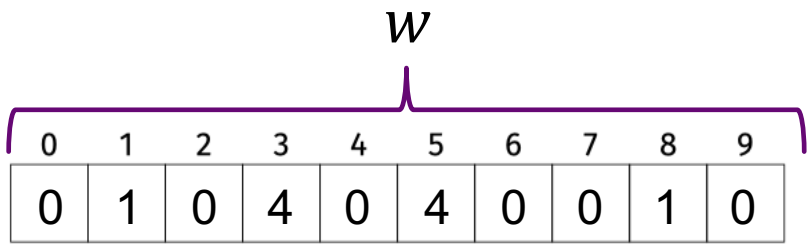
流式统计问题：频率统计

- **为什么要频率统计？**
  - 识别 **大流量用户 (Heavy Hitters)**，检测 **异常/攻击流量**，支持 **计费与网络优化**
- **难点**
  - 流数量巨大（10^7 ~ 10^9，内存受限，不能逐个存计数，必须线速处理（纳秒级更新）
- 给定一个集合$X = \{x_1, x_2, \dots, x_n\}$
  - 实现$Count(x)$：x出现的次数
  - 实现$increment(x)$：向集合中插入新/已有元素

基础方案：哈希表

- 方案：哈希表计数，键：IP 地址 / 关键词 / 消息 ID，值：出现次数
- 键空间极大 → 内存爆炸
  - 在流量统计里，键（Key）可能是 IP 地址（IPv4 有 2^{32} 种，IPv6 更是 2^{128}种），或消息 ID、关键词（几乎无穷大）。
- 大部分键很稀疏 → 浪费
  - 流量分布通常是长尾分布（Zipf 分布），少数“热点键”（Heavy Hitters）出现频率高，大量键只出现 1 次或很少几次。
- 高频更新 → 扩容成本高
  - 哈希表负载因子高时必须扩容，否则冲突严重、性能下降。
  - 扩容需要重新分配更大内存，并重新 rehash 所有已有键。
  - **在流量统计场景里，**数据包到达速度极快（纳秒级），一旦触发扩容，会带来明显性能抖动甚至无法承受。

哈希到计数器中



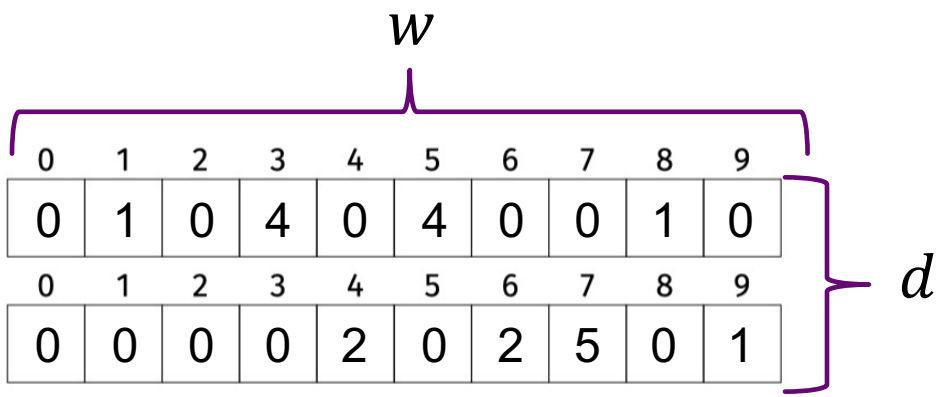
输入	位置
s	hash(s)
"surf"	3
"sand"	8
"data"	5
"sun"	1
"beach"	5

"data"
"surf"
"sand"
"surf"
"surf"
"beach"
"data"
"beach"
"surf"
"sun"

- 用长度为w的数组存储计数器($w << n$)
- **问题：**出现偏差(偏高)
  - 不同Key 映射 到 同一位置

Count-Min Sketch算法

- **思路：**用多个哈希函数



s	hash_1(s)	hash_2(s)
"surf"	3	7
"sand"	8	7
"data"	5	4
"sun"	1	9
"beach"	5	6

"data"
"surf"
"sand"
"surf"
"surf"
"beach"
"data"
"beach"
"surf"
"sun"

- 用长度为 w 的d个数组存储计数器，每个数组有独立的哈希函数
- **查询：**选择所有数组计数的**最小值**（减小误差）

- 空间复杂度：$O(wd)$
- 时间复杂度
  - 插入：$O(d)$，查询：$O(d)$

看 hash_1("surf") = 3，去上面数组第 3 格 → 计数 = 4。

看 hash_2("surf") = 7，去下面数组第 7 格 → 计数 = 5。

结果取最小值：min(4, 5) = 4。

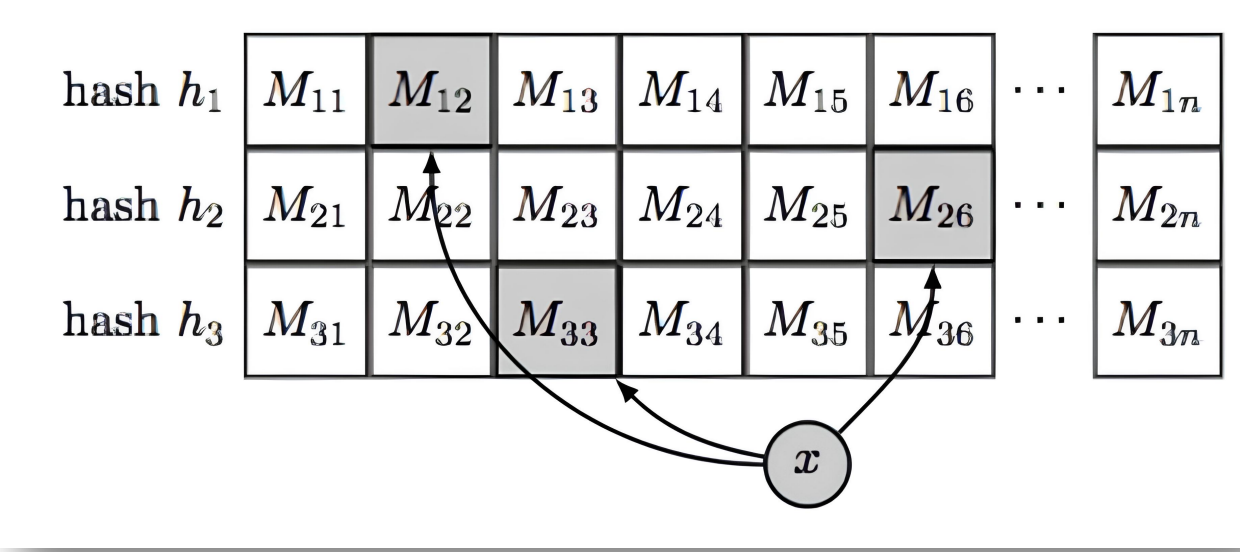
所以估计 "surf" 出现了 4 次。

Count-Min Sketch 算法

Algorithm 1 Count-Min Sketch

insert(x):
for $i = 1$ **to** d **do**
  $M[i, h_i(x)] \leftarrow M[i, h_i(x)] + 1$
end for

query(x):
$c = \min \{M[i, h_i(x)]$ for all $1 \leq i \leq d\}$
return c



例子


1 参数设置
行数 $d = 3$
每行宽度 $w = 5$ （桶编号 0～4）
3 个独立的哈希函数 h_1, h_2, h_3
所有计数器初始为 0
2 哈希函数（示例：固定映射）
元素 x | $h_1(x)$ | $h_2(x)$ | $h_3(x)$
A | 0 | 1 | 2
B | 1 | 0 | 3
C | 4 | 1 | 0
D | 3 | 4 | 0
3 数据流（依次到达）
A B A C B A D C A
时间顺序（共 9 个元素）
4 逐步处理数据流（每个元素同时更新 3 行）
初始状态
Step 1 处理 A
Step 2 处理 B
Step 3 处理 A
Step 4 处理 C
Step 5 处理 B
Step 6 处理 A
Step 7 处理 D
Step 8 处理 C
Step 9 处理 A
0 1 2 3 4
行 1 0 0 0 0 0
行 2 0 0 0 0 0
行 3 0 0 0 0 0
行 1 1 0 0 0 0
行 2 0 1 0 0 0
行 3 0 0 1 1 0
行 1 1 1 0 0 0
行 2 1 1 0 0 0
行 3 0 0 1 1 0
行 1 2 1 0 0 0
行 2 1 2 0 0 0
行 3 0 0 2 1 0
行 1 2 1 0 0 1
行 2 1 3 0 0 0
行 3 1 0 2 1 0
行 1 2 2 0 0 1
行 2 2 3 0 0 0
行 3 1 0 2 2 0
行 1 3 2 0 0 1
行 2 2 4 0 0 0
行 3 1 0 3 2 0
行 1 3 2 0 1 1
行 2 2 4 0 1 1
行 3 2 0 3 2 1
行 1 3 2 0 1 2
行 2 2 5 0 1 1
行 3 2 0 3 2 1
行 1 4 2 0 1 2
行 2 2 6 0 1 1
行 3 2 0 4 2 1
5 查询示例（读取 3 行对应桶，取最小值）
查询 A
$h_1(A)=0$ → 行1计数 = 4
$h_2(A)=1$ → 行2计数 = 6
$h_3(A)=2$ → 行3计数 = 4
$\hat{f}_A = \min(4, 6, 4) = 4$
真实频率 $f_A = 4$ （估计正确）
查询 B
$h_1(B)=1$ → 行1计数 = 2
$h_2(B)=0$ → 行2计数 = 2
$h_3(B)=3$ → 行3计数 = 2
$\hat{f}_B = \min(2, 2, 2) = 2$
真实频率 $f_B = 2$ （估计正确）
查询 C
$h_1(C)=4$ → 行1计数 = 2
$h_2(C)=1$ → 行2计数 = 5
$h_3(C)=0$ → 行3计数 = 2
$\hat{f}_C = \min(2, 5, 2) = 2$
真实频率 $f_C = 2$ （估计正确）
查询 D
$h_1(D)=3$ → 行1计数 = 1
$h_3(D)=4$ → 行2计数 = 1
$h_3(D)=0$ → 行3计数 = 1
$\hat{f}_D = \min(1, 1, 1) = 1$
真实频率 $f_D = 1$ （估计正确）
6 总结
Count-Min Sketch 只会高估，不会低估： $\hat{f}_x \geq f_x$。
查询时取 3 行计数的最小值，选择“碰撞最少”的一行。
误差来自哈希冲突（不同元素落到同一桶）。
本例中所有元素估计都正确，但一般情况下可能高估。
空间复杂度： $O(d \cdot w)$ 单次更新时间： $O(d)$。
误差保证（以至少 $1 - \delta$ 的概率成立）：
$\hat{f}_x \leq f_x + \varepsilon n$

误差率分析

- Count-Min Sketch 是概率数据结构：会返回有误差的结果
- 我们需要回答两个问题：
  - 误差大概范围是多少？
  - w 和 d 的选择对误差有什么影响？

符号定义

- **假设：**一个集合包含 n 个元素，需要进行频率估计
- 设：对任意一个元素 x，f_x 是其真实频率
- 第$i \in [d]$个哈希表中，对元素x的频率估计结果为$\hat{f}_x^{(i)} = M[i, h_i(x)]$
- 聚合后的估计为 $\hat{f}_x = \min_i\{\hat{f}_x^{(i)}\}$
- 关系满足：$\hat{f}_x^{(i)} \geq \hat{f}_x \geq f_x$

绝对误差和相对误差

- **绝对误差**

$$\hat{f}_x - f_x$$

  - 随着数据流增长，可能与 x发生 Hash 碰撞的机会增多，计数器可能被其他元素“抬高”，因此误差会累积

- **相对误差**

$$\frac{\hat{f}_x - f_x}{f_x}$$

- 对高频元素，若$f_x = \Theta(n)$，则控制相对误差，本质上等价于要求绝对误差相对于总流量 n 足够小：

$$\frac{\hat{f}_x - f_x}{n} < \epsilon \Rightarrow \boxed{\hat{f}_x - f_x < \epsilon n}.$$

为什么关注相对误差

- 在大规模数据流中，我们通常更关注 **heavy hitters**。定义频率阈值 $\phi > 0$，若$f_x \geq \phi n$，则称 x为 ϕ-heavy hitter。
- Count-Min Sketch（CMS） 对任意元素提供加性误差保证：

$$0 \leq \hat{f}_x - f_x \leq \epsilon n$$

- （以至少 $1 - \delta$的概率成立）。
- 对 heavy hitter，由于 $f_x \geq \phi n$，因而其相对误差满足$\frac{\hat{f}_x - f_x}{f_x} \leq \frac{\epsilon n}{\phi n} = \boxed{\frac{\epsilon}{\phi}}$
- 因此，当 $\epsilon \ll \phi$时，CMS 对高频元素可以提供有意义的相对精度，并支持 heavy hitter 的识别。

例子

- **对低频元素（non-heavy hitter）的影响**
  - 假设总流量 $n = 10000$，$\epsilon = 0.01$，则 CMS 以至少 $1 - \delta$的概率满足：$0 \le \hat{f}_y - f_y \le \epsilon n = 100$
  - 若某个低频元素真实频次为$f_y = 20$，则$\hat{f}_y \le 120$
  - 因而其相对误差上界为 $\frac{\hat{f}_y - f_y}{f_y} \le \frac{100}{20} = 500\%$
  - **因此，对低频元素，CMS 的相对误差可能非常大。**
- **对热点元素（heavy hitter）的影响**
  - 仍取 $n = 10000, \epsilon = 0.01$，误差上界为 100。若某热点元素真实频次为$f_x = 5000$，则$\hat{f}_x \le 5100$
  - 相对误差上界为$\frac{\hat{f}_x - f_x}{f_x} \le \frac{100}{5000} = 2\%$
  - **因此，相同的加性误差对高频元素造成的相对影响很小。**

$\hat{f}_x^{(i)}$的期望

- 在分析 **Count-Min Sketch** 的性能时，**默认假设哈希函数是均匀且独立的**
- $E[\hat{f}_x^{(i)}]$：第$i \in [d]$个哈希表中针对x的频率期望

$$E[\hat{f}_x^{(i)}] = f_x + E(\text{所有y} \neq x\text{冲突的频率}) = f_x + \sum_{y \neq x} f_y * \Pr(h_i(x) = h_i(y))$$

  - $= f_x + \frac{1}{w}\sum_{y \neq x} f_y \leq f_x + \frac{n}{w} \Longrightarrow E[\hat{f}_x^{(i)} - f_x] \leq \frac{n}{w}$

- 设$f_x = p_x n, p_x \in (0,1]$，$\boxed{E\left[\frac{\hat{f}_x^{(i)} - f_x}{f_x}\right] \leq \frac{n/w}{p_x n} = \frac{1}{p_x w}}$，**相对误差的期望和n无关！**

$\hat{f}_x^{(i)}$的误差

- 设第i行对 x的碰撞误差为 $X_i = \hat{f}_x^{(i)} - f_x$
- 因为 CMS 只会高估，所以 $X_i \geq 0$，前面已经得到：$E[X_i] = E\left[\hat{f}_x^{(i)} - f_x\right] \leq \frac{n}{w}$
- 由 Markov 不等式：$\Pr(X_i \geq \alpha) \leq \frac{E[X_i]}{\alpha} \leq \frac{n}{w\alpha}$
- 现在希望： $\Pr(X_i \geq \alpha) \leq \frac{1}{2}$，只需要让 Markov 给出的上界不超过 $1/2$：$\frac{n}{w\alpha} \leq \frac{1}{2}$
- 解得：$\boxed{\alpha \geq \frac{2n}{w}}$，因此取最自然的阈值$\boxed{\alpha = \frac{2n}{w}}$，就得到：
- $\boxed{\Pr\left(\hat{f}_x^{(i)} - f_x \geq \frac{2n}{w}\right) \leq \frac{1}{2}}$

$\hat{f}_x$的误差

- 如果最小值 $\geq \frac{2n}{w}$，则要求每个哈希表（总共d个）中都 $\geq \frac{2n}{w}$
- 这种情况发生概率（独立）：

$$\prod_{i=1}^{d} \Pr(\hat{f}_x^{(i)} - f_x \geq \frac{2n}{w}) \leq \left(\frac{1}{2}\right)^d \Rightarrow \Pr\left(\hat{f}_x - f_x \geq \frac{2n}{w}\right) \leq \left(\frac{1}{2}\right)^d$$

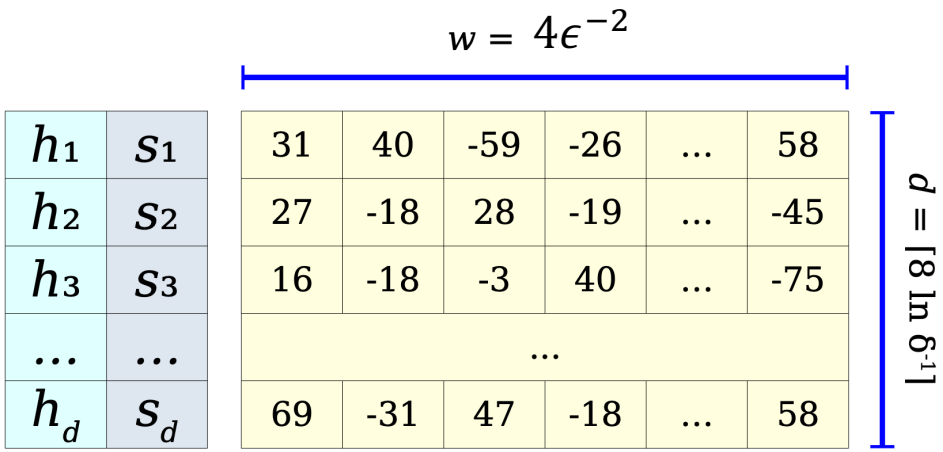
- 若$\Pr(\hat{f}_x - f_x \geq \epsilon n) \leq \left(\frac{1}{2}\right)^d$，只需要：$\frac{2n}{w} \leq \epsilon n$，约掉 n：$\frac{2}{w} \leq \epsilon$，得到：$\boxed{w \geq \frac{2}{\epsilon}}$
- 误差边界：$\left(\frac{1}{2}\right)^d = \delta \Rightarrow d = \log_2 \frac{1}{\delta}$

参数相关的结论

- 当$d = \Omega\left(\log\frac{1}{\delta}\right), w \geq \frac{2}{\epsilon}$时，针对集合 X 里的任意元素 x_i，其频率估计 $\hat{f}_{x_i}$ 和真实评率 f_{x_i} 满足：

$$\Pr(\hat{f}_{x_i} \geq f_{x_i} + \epsilon n) \leq \delta$$

Count-Sketch 算法




```
increment(x):
   for i = 1 … d:
      count[i][hᵢ(x)] += sᵢ(x)
```

```
estimate(x):
   options = []
   for i = 1 … d:
      options += count[i][hᵢ(x)] * sᵢ(x)
   return medianOf(options)
```

Count Sketch 多了一个非常关键的东西：**Sign Hash**。

每一行有两个 Hash：$h_i(x) \to [w]$

以及：

$$s_i(x) \to \{-1, +1\}.$$

更新时：

$$\boxed{C_i[h_i(x)] \mathrel{+}= s_i(x)}$$

$\hat{f}_x^{(i)}$的计算

- 不同于Count-Min Sketch, $\hat{f}_x^{(i)}$不仅和其他元素的分布有关还和哈希函数$s_i(x)$有关
- Count Sketch 第i行中，对x的单行估计为$\hat{f}_x^{(i)} = s_i(x)\, C[i, h_i(x)]$
- 而桶 $h_i(x)$中包含所有与 x冲突的元素：$C[i, h_i(x)] = s_i(x) f_x + \sum_{\substack{y \neq x \\ h_i(y)=h_i(x)}} s_i(y) f_y.$
- 因此

$$\hat{f}_x^{(i)} = s_i(x)\left[s_i(x) f_x + \sum_{\substack{y \neq x \\ h_i(y)=h_i(x)}} s_i(y) f_y\right]$$

$$= f_x + \sum_{\substack{y \neq x \\ h_i(y)=h_i(x)}} s_i(x) s_i(y) f_y,$$

- 因$s_i(x)^2 = 1$，所以单行估计误差：$\boxed{\hat{f}_x^{(i)} - f_x = \sum_{\substack{y \neq x \\ h_i(y)=h_i(x)}} s_i(x) s_i(y) f_y}$

$\hat{f}_x^{(i)}$的期望

- 定义一个指示器随机变量$X_y^{(i)}$

$$X_y^{(i)} = \begin{cases} 1 & if\ \mathrm{h}_i(x) = \mathrm{h}_i(y) \\ 0 & if\ \mathrm{h}_i(x) \neq \mathrm{h}_i(y) \end{cases}$$

- 则估计的随机变量为$\hat{f}_x^{(i)} = f_{\mathrm{x}} + \sum_{y \neq x} \mathrm{s}_i(x)\mathrm{s}_i(y) f_y X_y^{(i)}$
- $E[\mathrm{s}_i(x)] = \frac{1}{2} * -1 + \frac{1}{2} * 1 = 0$
- $E[\hat{f}_x^{(i)}] = E\left[f_{\mathrm{x}} + \sum_{y \neq x} \mathrm{s}_i(x)\mathrm{s}_i(y) f_y X_y^{(i)}\right] = E[f_{\mathrm{x}}] + E\left[\sum_{y \neq x} \mathrm{s}_i(x)\mathrm{s}_i(y) f_y X_y^{(i)}\right] = f_x + \sum_{y \neq x} E\left[\mathrm{s}_i(x)\mathrm{s}_i(y) f_y X_y^{(i)}\right] = f_x + \sum_{y \neq x} E[\mathrm{s}_i(x)\mathrm{s}_i(y)]\, E\left[f_y X_y^{(i)}\right] = f_x + \sum_{y \neq x} E[\mathrm{s}_i(x)] E[\mathrm{s}_i(y)]\, E\left[f_y X_y^{(i)}\right] = f_x$

CS和CMS的区别

- CMS 的估计是 **有偏的（总是高估）**。
- CS 的估计是 **无偏的（期望正好等于真实值）**。

- CMS 的估计具有**单边误差**：碰撞只会导致高估。
- CS 的估计具有**双边误差**：可能高估，也可能低估，但期望等于真实值。
- 因此，CMS 更适合需要“**不低估**”保证的场景；CS 更适合关注**无偏频率估计**及平方误差的场景。

- **“宁可多报，不要漏报”**：CMS 往往更合适。
- **“希望估计围绕真实值波动，不系统性偏高”**：CS 更合适。
- **数据里有少数特别大的 heavy hitters，对其他元素污染很强**：CS 往往表现更好，因为正负噪声可以抵消。

$\hat{f}_x^{(i)}$的误差

- $\hat{f}_x^{(i)}$和f_x差多少？ **$\hat{f}_x^{(i)}$-f_x可能为正数或负数**
- 切比雪夫不等式：对随机变量X，存在$c > 0$，满足

$$\Pr[|X - E[X]| > c] \leq \frac{\boldsymbol{Var[X]}}{c^2}$$

- $X = \hat{f}_x^{(i)}$，$E[X] = f_x$
- 如果能够计算**方差**，就可以用切比雪夫不等式给出：偏离真实值超过 c 的概率不会超过 $\frac{Var[X]}{c^2}$
- 这就限定了“不良估计”发生的概率。

方差估计

- 在第 i 行，Count-Sketch 对元素 x的估计是

$$\hat{f}_x^{(i)} = f_x + \sum_{y \neq x} s_i(x)\, s_i(y)\, f_y\, X_y^{(i)}.$$

- 记误差为

$$\varepsilon_i = \hat{f}_x^{(i)} - f_x = \sum_{y \neq x} Z_y\,,$$

- 其中

$$Z_y = s_i(x)\, s_i(y)\, f_y\, X_y^{(i)}.$$

方差估计

- 已知$\varepsilon_i = \hat{f}_x^{(i)} - f_x = \sum_{y \neq x} Z_y$，
- 那么根据和的方差公式：

$$\boxed{\mathrm{Var}(\varepsilon_i) = \mathrm{Var}\left(\sum_{y \neq x} Z_y\right) = \sum_{y \neq x} \mathrm{Var}(Z_y) + 2 \sum_{\substack{y<z \\ y,z \neq x}} \mathrm{Cov}(Z_y, Z_z)}$$

- **方差的定义**
  - 对任意随机变量 A，

$$\mathrm{Var}[A] = \mathbb{E}[A^2] - \left(\mathbb{E}[A]\right)^2.$$

  - 如果 $A = U + V$，那么

$$\mathrm{Var}[U + V] = \mathrm{Var}[U] + \mathrm{Var}[V] + 2\mathrm{Cov}(U, V).$$

  - 推广到多个随机变量 $Z_1, Z_2, \ldots, Z_k$，就有

$$\mathrm{Var}\left(\sum_{i=1}^{k} Z_i\right) = \sum_{i=1}^{k} \mathrm{Var}\left[Z_i\right] + 2 \sum_{1 \leq i < j \leq k} \mathrm{Cov}\left(Z_i, Z_j\right).$$

回顾知识

- $\mathrm{Var}(U+V)=\mathbb{E}[(U+V)^2]-(\mathbb{E}[U+V])^2$
- 出发，展开得到

$$\mathbb{E}[U^2]+2\mathbb{E}[UV]+\mathbb{E}[V^2]-\left((\mathbb{E}[U])^2+2\mathbb{E}[U]\mathbb{E}[V]+(\mathbb{E}[V])^2\right),$$

- 再重新分组为

$$(\mathbb{E}[U^2]-(\mathbb{E}[U])^2)+(\mathbb{E}[V^2]-(\mathbb{E}[V])^2)+2(\mathbb{E}[UV]-\mathbb{E}[U]\mathbb{E}[V]),$$

- 于是

$$\boxed{\mathrm{Var}(U+V)=\mathrm{Var}(U)+\mathrm{Var}(V)+2\,\mathrm{Cov}(U,V)}$$

- 如果U,V独立，那么$\mathrm{Cov}(U,V)=0$，可以进一步简化成

$$\mathrm{Var}(U+V)=\mathrm{Var}(U)+\mathrm{Var}(V)\,.$$

单项期望与方差

- 定义：$Z_y = s_i(x)\, s_i(y)\, f_y\, X_y^{(i)}$ （$X_y^{(i)} = \mathbf{1}[h_i(y) = h_i(x)]$）
- **期望：**

$$\mathbb{E}[s_i(x)s_i(y)] = 0 \;\Rightarrow\; \mathbb{E}\left[Z_y\right] = 0.$$

- **方差：**

$$\mathrm{Var}\left[Z_y\right] \;=\; \mathbb{E}\left[Z_y^2\right] \;=\; f_y^2\, \mathbb{E}\left[X_y^{(i)}\right] \;=\; f_y^2\, \Pr\left[h_i(y) = h_i(x)\right]$$

- 若每行有 w 个桶，**且位置哈希均匀**，冲突概率是 $1/w$：

$$\mathrm{Var}\left[Z_y\right] = \frac{f_y^2}{w}.$$

- **参考：$\boldsymbol{Var[Z] = E\left[(Z - E[Z])^2\right] = E\left[Z^2\right] - E[Z]^2}$**

协方差

- 对 $y \neq z$，$\mathrm{Cov}(Z_y, Z_z) = \mathbb{E}[Z_y Z_z] - \mathbb{E}[Z_y]\mathbb{E}[Z_z]$
- 由于 $\mathbb{E}[Z_y] = 0$，所以$\mathrm{Cov}(Z_y, Z_z) = \mathbb{E}[Z_y Z_z]$.
- 展开：

$$\begin{aligned}\mathbb{E}[Z_y Z_z] &= f_y f_z \, \mathbb{E}\left[s_i(y) s_i(z) X_y^{(i)} X_z^{(i)}\right] \\ &= f_y f_z \, \mathbb{E}[s_i(y) s_i(z)] \mathbb{E}\left[X_y^{(i)} X_z^{(i)}\right]\end{aligned}$$

- 取期望：
  - $s_i(y), s_i(z)$独立且均匀分布在 $\{\pm 1\}$，
  - 所以 $\mathbb{E}[s_i(y) s_i(z)] = 0$，
  - 因此 $\mathbb{E}[Z_y Z_z] = 0$。
- 结论：

$$\mathrm{Cov}(Z_y, Z_z) = 0.$$

方差估计

- **最终结果**

$$\mathrm{Var}[\varepsilon_i] = \sum_{y \neq x} \frac{f_y^2}{w}.$$

- 即

$$\mathrm{Var}\left[\hat{f}_x^{(i)}\right] = \frac{1}{w} \sum_{y \neq x} f_y^2$$

- 上界是

$$\frac{1}{w} \sum_{y \neq x} f_y^2 = \frac{\|f\|_2^2 - f_x^2}{w} \leq \frac{\|f\|_2^2}{w} \quad (\|f\|_2^2 = \sum_y f_y^2)$$

$\hat{f}_x^{(i)}$的误差

- 令 $c = \epsilon\|f\|_2$，我们有

$$\Pr[|X - E[X]| > c] \leq \frac{Var[X]}{c^2} \Rightarrow \Pr\left[\left|\hat{f}_x^{(i)} - f_x\right| > \epsilon\|f\|_2\right] \leq \frac{Var\left[\hat{f}_x^{(i)}\right]}{(\epsilon\|f\|_2)^2}$$

- $Var\left[\hat{f}_x^{(i)}\right] \leq \frac{\|f\|_2^2}{w} \Rightarrow \Pr\left[\left|\hat{f}_x^{(i)} - f_x\right| > \epsilon\|f\|_2\right] \leq \frac{\|f\|_2^2}{w}\frac{1}{(\epsilon\|f\|_2)^2} = \frac{1}{w\epsilon^2}$
- $w = 4\epsilon^{-2} \Rightarrow \Pr\left[\left|\hat{f}_x^{(i)} - f_x\right| > \epsilon\|f\|_2\right] \leq \frac{1}{4}$

- 一行 Count Sketch 有至少$\boxed{\frac{3}{4}}$的概率满足$\left|\hat{f}_x^{(i)} - f_x\right| < \epsilon\|f\|_2$.
- $\boxed{w = \Theta(1/\epsilon^2) \Rightarrow \text{单行以常数概率获得 } \epsilon\|f\|_2 \text{ 的加性误差}}$

$\hat{f}_x$的误差

- CS输出是中位数估计，以很高概率得到误差在允许范围内的估计值
- 直觉原因：
  - 中位数不在“好的”区域的唯一情况是，**至少有一半的估计值位于“坏的”（偏离很大）区域**
  - 考虑到每个单独的哈希表都有“一定”的概率是好的（之前分析），所以这种情况非常不可能发生。
- 令 D 为 一个随机变量，表示产生的估计与真实的误差超过$\epsilon\|f\|_2$的哈希表的数量（**坏的结果**）。
- $D=\sum_{i=1}^{d} I_i$,其中$I_i=\begin{cases}1, & \text{第 } i \text{ 行 坏}\\ 0, & \text{第 } i \text{ 行 好}\end{cases}$，满足$\Pr(I_i=1)=p_i\le\frac{1}{4}$
- 我们可以构造一个“更坏的假想情况”：假设每一行坏的概率都直接取最大值$1/4$。令

$$B \sim \mathrm{Bin}(d, 1/4)\text{（二项分布）}$$

一共做 d次独立实验，每次有 $1/4$的概率“失败”，B表示最后一共失败了多少次。

- 那么这个假想的B会比真实的D更容易出现很多坏行。
- 所以对于任意阈值 t：$\boxed{\Pr(D\ge t)\le\Pr(B\ge t)}$，也就是$\boxed{B \text{ 随机支配 } D}$

问题转换

- 在 d次独立实验中，每次实验以概率 $1/4$得到“坏行”，那么 D 表示“坏行的总数”，它的分布就是二项分布。

$$\Pr\left(\left|\hat{f}_x - f_x\right| \geq \epsilon \|f\|_2\right) \leq \Pr(D \geq d/2) \leq \Pr(B \geq d/2).$$

- 如果$X \sim \mathrm{Bin}(n, p)$且$p < \frac{1}{2}$，由 Hoeffding 不等式，则

$$\Pr\left[X \geq \frac{n}{2}\right] \leq e^{-n \cdot z(p)}, \quad z(p) = \frac{\left(\frac{1}{2} - p\right)^2}{2p}$$

- 这里$B \sim \mathrm{Bin}(d, 1/4)$，带入得

$$\Pr\left[D \geq \frac{d}{2}\right] \leq e^{-d \cdot z\left(\frac{1}{4}\right)} = e^{-d/8}$$

- 当$d = 8\ln(\frac{1}{\delta})$时，

$$\Pr\left(\hat{f}_x - f_x \geq \epsilon \|f\|_2\right) \leq \Pr\left[D \geq \frac{d}{2}\right] \leq \delta$$

参数相关的结论

- 当

$$w \geq 4\epsilon^{-2}, d \geq 8\ln(1/\delta)$$

- 时，对于任意固定元素 x_i，Count Sketch 的频率估计满足

$$\Pr\left[\left|\hat{f}_{x_i} - f_{x_i}\right| \geq \epsilon \|f\|_2\right] \leq \delta$$

额外知识：KV Cache

- **大语言模型中的自回归生成（Autoregressive Generation in LLM）**
  - LLM 每次只生成一个 token，并根据前面已经生成的 token 来预测下一个 token。
  - 每生成一个 token，就将该 token 追加到当前序列中，并继续生成下一个 token，直到生成结束符 **EOS**

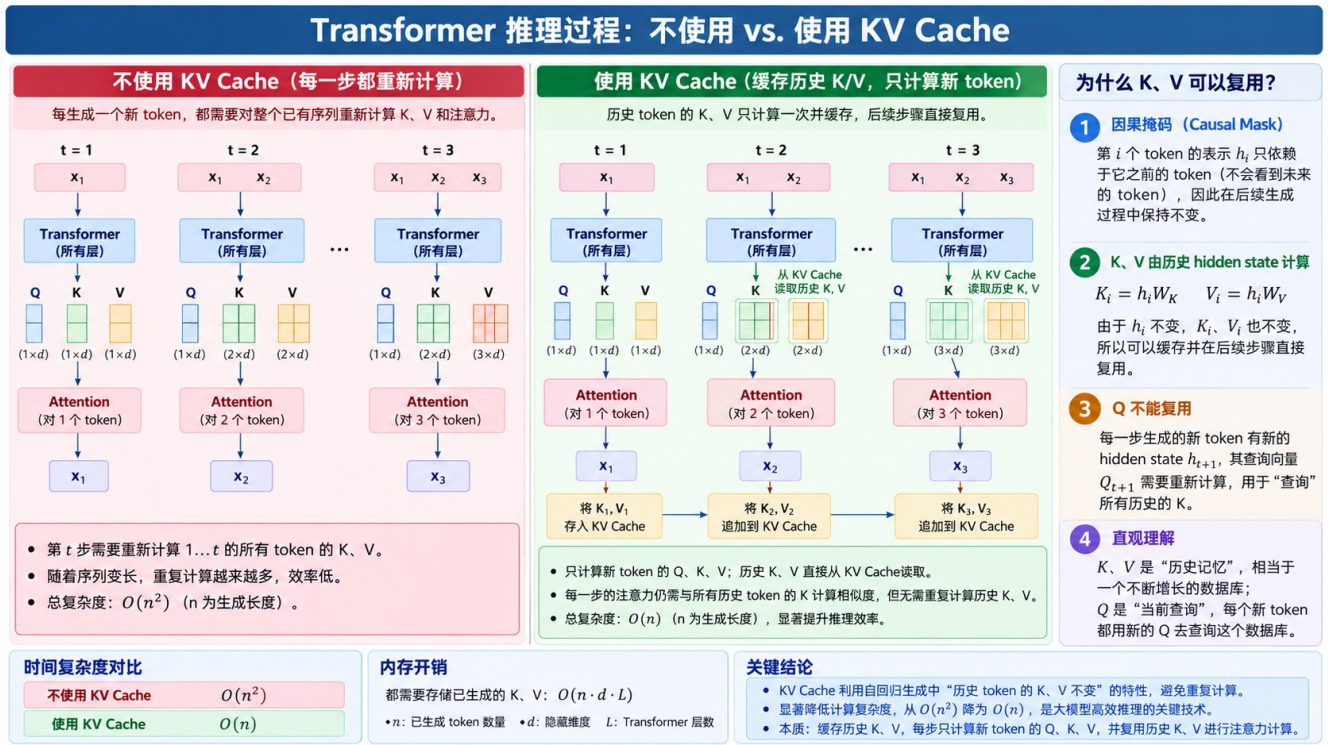
```
step 0 input: Lionel Messi is a player
step 1 input: Lionel Messi is a player who
step 2 input: Lionel Messi is a player who has
step 3 input: Lionel Messi is a player who has been
step 4 input: Lionel Messi is a player who has been a
step 5 input: Lionel Messi is a player who has been a key
step 6 input: Lionel Messi is a player who has been a key part
step 7 input: Lionel Messi is a player who has been a key part of
step 8 input: Lionel Messi is a player who has been a key part of the
step 9 input: Lionel Messi is a player who has been a key part of the team
step 10 input: Lionel Messi is a player who has been a key part of the team's
step 11 input: Lionel Messi is a player who has been a key part of the team's success
step 12 input: Lionel Messi is a player who has been a key part of the team's success.
step 13 input: Lionel Messi is a player who has been a key part of the team's success.

 Input: Lionel Messi is a
Output: Lionel Messi is a player who has been a key part of the team's success.
```



额外知识：KV Cache

- **目标**：通过缓存之前计算得到的 Key 和 Value，我们只需要为新生成的 token 计算注意力，因为此前所有 token 对应的 K 和 V 都保持不变。



Attention 是：

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V.$$

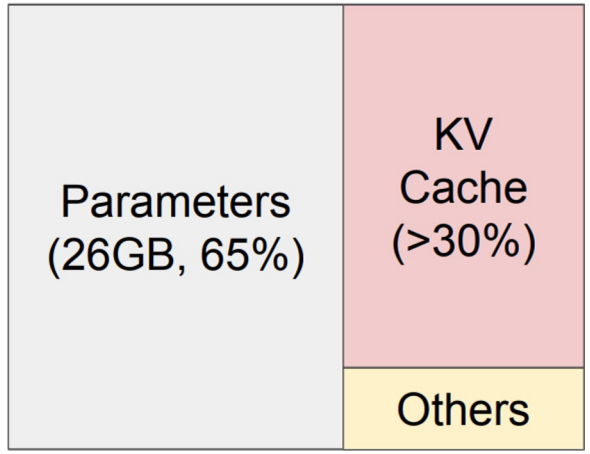
在第$t+1$步，我们真正需要的是：Q_{t+1}去和$K_1, K_2, \ldots, K_t, K_{t+1}$做点积。

也就是：$Q_{t+1}\begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_{t+1} \end{bmatrix}^T$，然后得到 attention weights，再乘：$V_1, V_2, \ldots, V_{t+1}$.

所以历史的 K/V 是被“查询”的数据库，而新的 Q 是这一步新产生的查询向量

挑战

- **大语言模型推理的主要瓶颈：**
  - **模型规模大**
  - **Attention 层具有二次复杂度开销**
  - **KV Cache 占用大量显存**
    - 在生成过程中，需要存储 Attention 的中间 **Key 和 Value**
    - KV Cache 会带来巨大的显存开销
    - 例如：**30B 模型，Batch Size = 128，Sequence Length = 1024**



13B LLM on A100-40GB

180GB KV Cache!!!

- **挑战：如何在不降低模型性能的前提下，减少 LLM 推理过程中的 KV Cache 显存占用？**

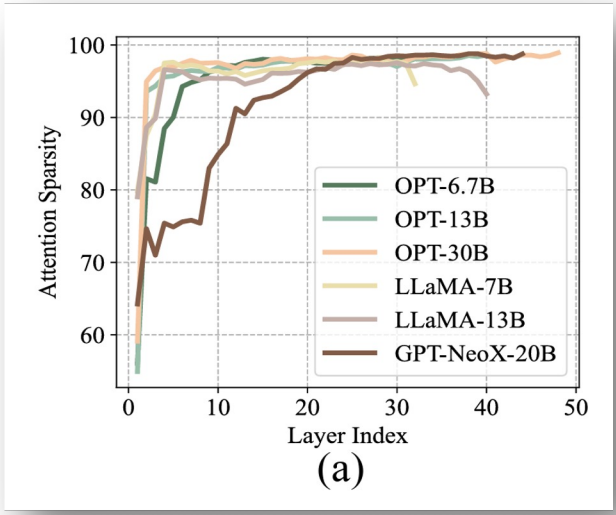
设计初衷

- **理想的 KV Cache 应具备：**
- **较小的缓存规模**：降低显存占用
- **较低的 Cache Miss Rate**：保证模型性能和长上下文生成准确率
  - 被淘汰的 KV embedding **之后将无法再次访问**
  - 因此必须谨慎决定**淘汰哪些 KV**
- **低开销的淘汰策略（Eviction Policy）**：减少额外计算开销和实际推理时间（Wall-clock Time）

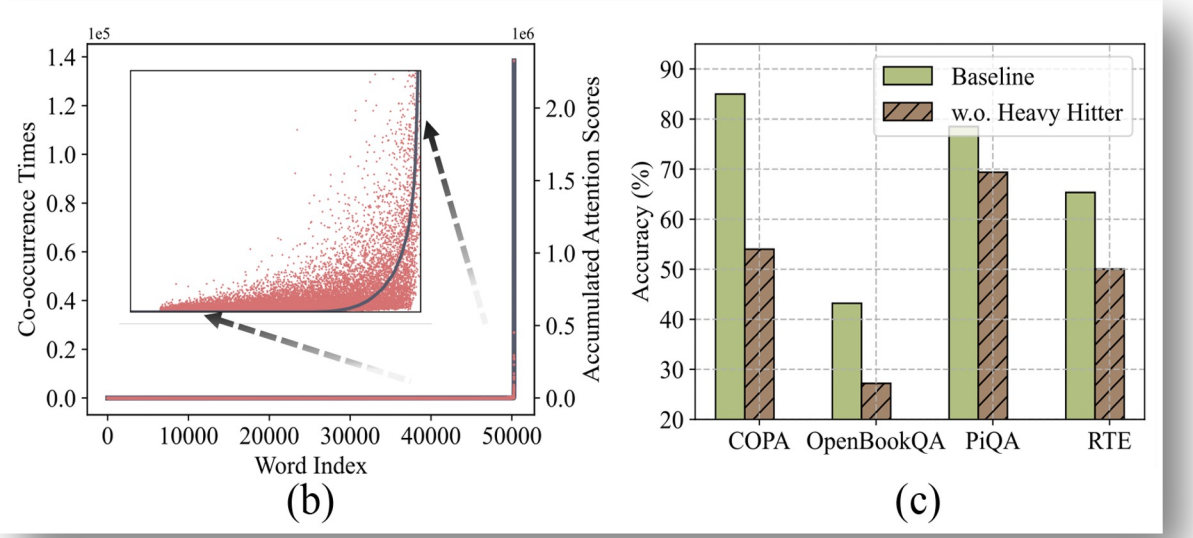
目标：在不牺牲生成性能的前提下，提高大语言模型推理的吞吐量

方案设计：观察

- **利用稀疏性减小 Cache 大小**
  - **Attention 稀疏性**
    - 若某个 Attention 值小于最大 Attention 值的 **1%**，则将其视为稀疏项
    - 尽管 LLM 采用稠密训练，**Attention Score Matrix 仍然高度稀疏**
  - 几乎所有层的 Attention 稀疏度都超过 **95%**
  - 这意味着：**生成下一个 token 时，并不一定需要访问所有历史 Key 和 Value**

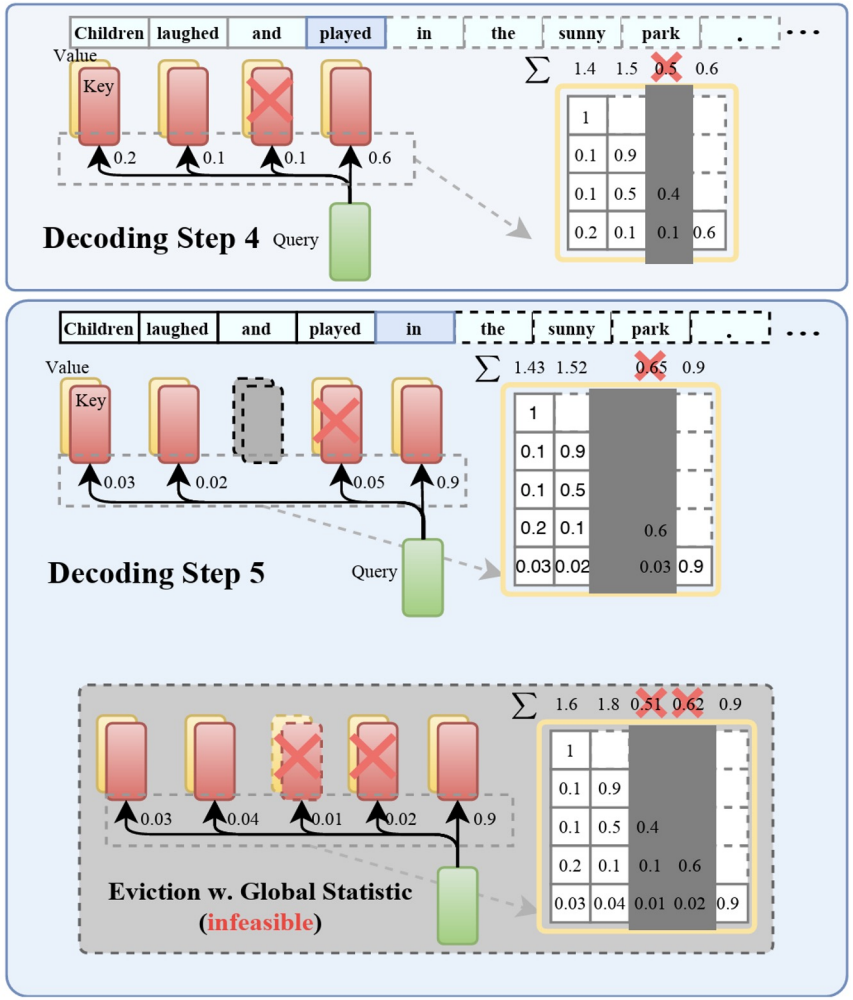


- **利用 Heavy Hitters 降低 Cache Miss Rate**
  - **累积 Attention Score 呈幂律分布（Power-law Distribution）**
  - 说明在生成过程中：**只有少量 token 持续具有较高的重要性**
  - 这些关键 token 被称为 **Heavy Hitters**
  - 如果将 Heavy Hitters 从 KV Cache 中淘汰：
  - **会显著破坏 LLM 的生成性能**



算法设计

- 算法最核心可以浓缩成一句话：
  KV Cache 只保留：Recent Tokens + Heavy Hitters
- 也就是两类 token：
  - **最近刚出现的 token**：先别急着删，因为它们还没来得及积累 attention。
  - **历史上累计 Attention 很高的 token**：说明它们一直重要，是 Heavy Hitters，尽量保留。
  - 例子：
    - 假设 **KV Cache 的容量预算为 3**。
    - 在完成第 4 个解码步骤后，根据**累积 Attention Score**，与第 3 个 token 对应的 KV embedding 会被淘汰。
    - 因此，这些被淘汰的 KV embedding 在后续解码步骤中将**无法再被访问**
- **为什么贪心策略可行？**
  - 每一步只根据**当前累计 Attention** 选择 Heavy Hitters
  - 实验发现，这种 **算法** 与利用未来 Attention 得到的全局选择效果相近
  - 因此，无需预测未来 token，也能有效识别应保留的关键 KV



实验结果

- 在不同模型规模和序列长度下均显著提升推理吞吐量（tokens/s）

Seq. length	512+32		512+512		512+1024	
Model size	6.7B	30B	6.7B	30B	6.7B	30B
Accelerate	20.4 (2, G)	0.6 (8, C)	15.5 (1, G)	0.6 (8, C)	5.6 (16, C)	0.6 (8, C)
DeepSpeed	10.2 (16, C)	0.6 (4, C)	9.6 (16, C)	0.6 (4, C)	10.1 (16, C)	0.6 (4, C)
FlexGen	20.2 (2, G)	8.1 (144, C)	16.8 (1, G)	8.5 (80, C)	16.9 (1, G)	7.1 (48, C)
H_2O (20%)	35.1 (4, G)	12.7 (728, C)	51.7 (4, G)	18.83 (416, C)	52.1 (4, G)	13.82 (264, C)

参考文献：Zhang, Z. et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.” NeurIPS, 2023.

哈希的另一种用法：相似性与检索

- 普通 Hash：尽量让相似的数据也分开
- 有没有一种Hash：能故意让相似的数据更容易撞在一起
- 数据库里有 1 亿张图片，给一张查询图片，找“最相似”的图片。



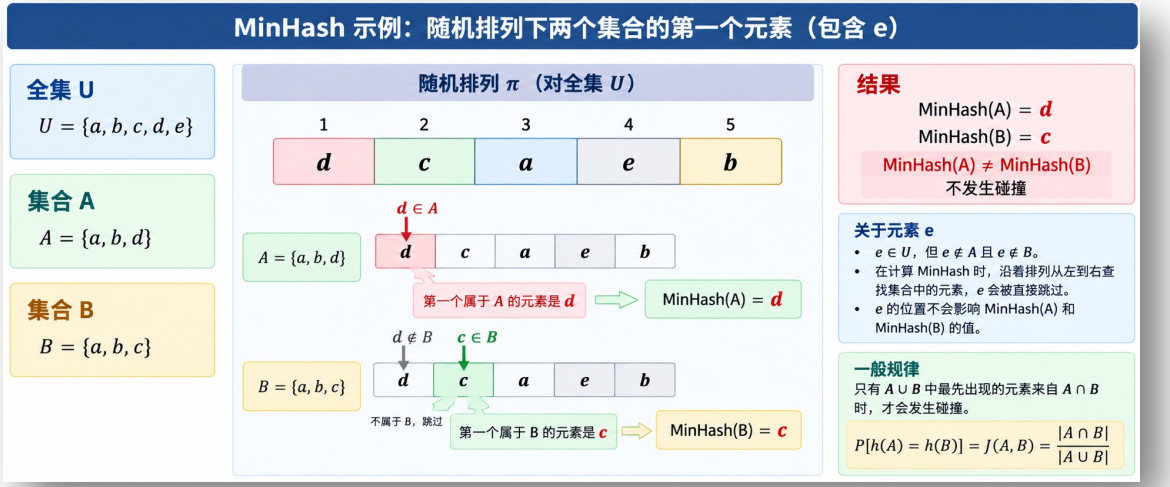
MinHash：让相似集合更容易发生 Hash Collision

- Jaccard 相似度：$J(A,B)=\frac{|A\cap B|}{|A\cup B|}$
- 能不能设计一个 Hash，使得两个集合越相似，Hash 值越容易相同？

- MinHash 最关键性质：$\boxed{P[h(A)=h(B)]=J(A,B)}$
- 例如：$J(A,B)=0.8\Rightarrow P[h(A)=h(B)]=0.8$

MinHash 的设计

- 算法：随机打乱（用哈希），再取第一个元素**（A 和 B 必须使用同一个随机排列！换句话说，同一个随机哈希就行）**
- 分析：$A \cup B$对于上面的例子：$A \cup B = \{a, b, c, d\}$，随机排列以后，在 $A \cup B$中排在最前面的元素，有两种情况。
- **情况 1：最前面的元素属于 $A \cap B$**
  - 这里：$A \cap B = \{a, b\}$，如果第一个元素是 a或 b，那么它同时属于 A 和 B。所以：$h(A) = h(B)$
- **情况 2：最前面的元素只属于其中一个集合**
  - 例如是：$c \in B \setminus A$ 或 $d \in A \setminus B$，那么： $h(A) \neq h(B)$.
  - 所以：$\boxed{h(A) = h(B) \Leftrightarrow A \cup B \text{ 中排列最靠前的元素属于 } A \cap B}$
- 而由于排列是随机的，$A \cup B$中每个元素成为“第一个元素”的概率相同。
- 因此：$P[h(A) = h(B)] = \frac{|A \cap B|}{|A \cup B|}$
- 于是：$\boxed{P[h(A) = h(B)] = J(A, B)}$



重复 MinHash，得到 Signature

- 做 k个独立 MinHash：$h_1, h_2, \ldots, h_k$
- 得到集合 A的 MinHash signature：$\boxed{\mathrm{Sig}(A) = [h_1(A), h_2(A), \ldots, h_k(A)]}$
- 就是对 Jaccard similarity 的估计：$\boxed{\hat{J}(A,B) = \frac{\#\{i : h_i(A) = h_i(B)\}}{k}}$



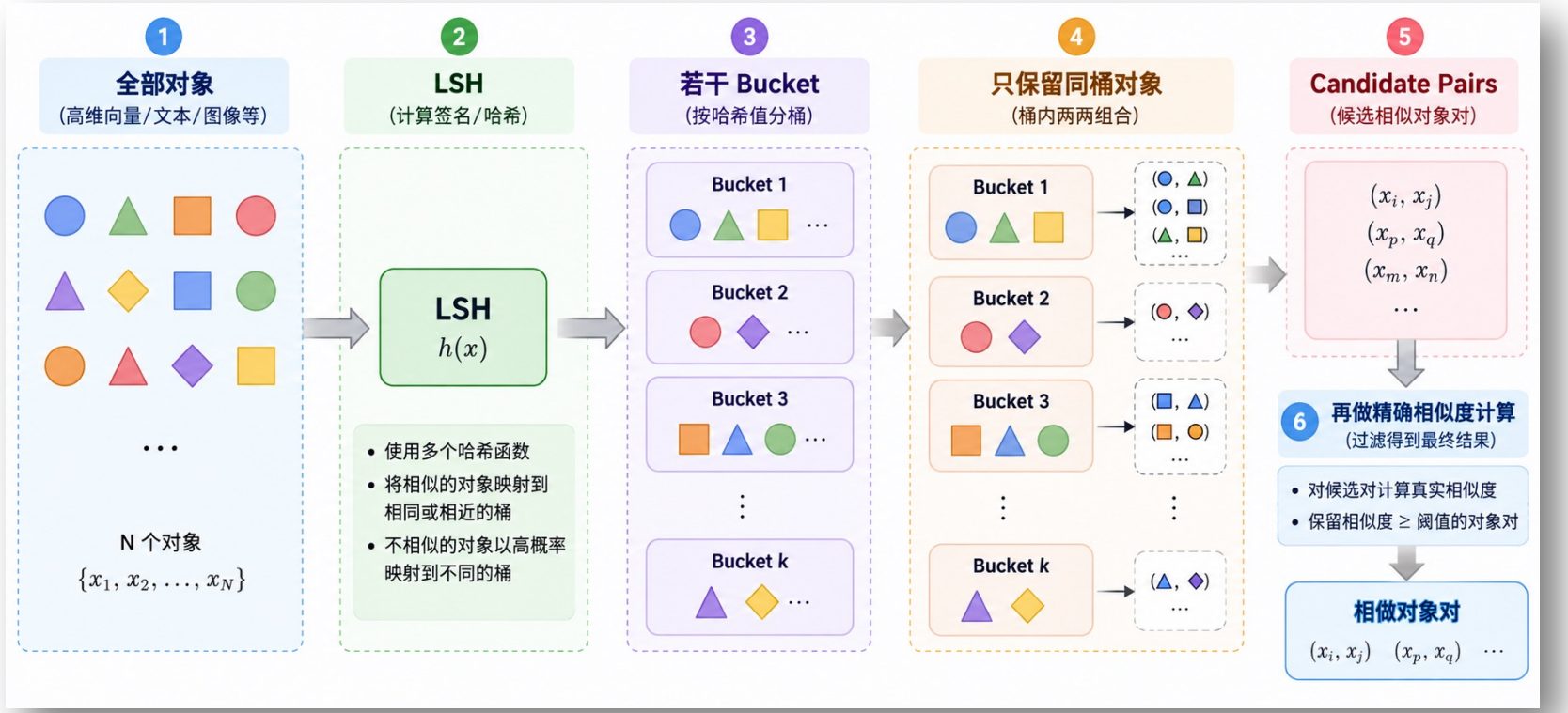
当N很大

- 假设有：$N = 10^8$个对象。
- 即使每个对象已经压缩成 MinHash signature，如果还是两两比较：
- $\binom{N}{2} = O(N^2)$.
- 所以问题变成：

MinHash 降低了“单次比较成本”， 但没有解决“候选对象太多”的问题

LSH：让相似对象进入同一个候选桶

- Local Sensitive Hashing（LSH）
- LSH 的目标不是直接判断“是否相似”， 而是快速排除绝大多数“不可能相似”的对象。



LSH设计思路

- 假设一个对象的 MinHash signature 长度是$k = br$
  - b：分成多少个 band（组）
  - r：每个 band 里有多少个 MinHash 值
  - 例如 $k = 6$，取 $b = 2,\quad r = 3$

$$\text{Sig}(A) = [3,7,2,\ 9,5,1]$$
被分成：$\text{Band } 1 = [3,7,2]$，$\text{Band } 2 = [9,5,1]$

$$\text{Sig}(B) = [3,7,2,\ 9,8,4]$$
也分成：$\text{Band } 1 = [3,7,2]$，$\text{Band } 2 = [9,8,4]$

- 对于 Band 1：$[3,7,2] = [3,7,2]$，三个位置全部相同，所以把这一整组$(3,7,2)$作为一个整体再 Hash：
  $$H(3,7,2) \rightarrow \text{某个 bucket}$$
- A 和 B 因为 band 内容完全相同，所以一定会被 Hash 到同一个 bucket

只要至少一个 band 碰撞，就把这两个对象列为候选对

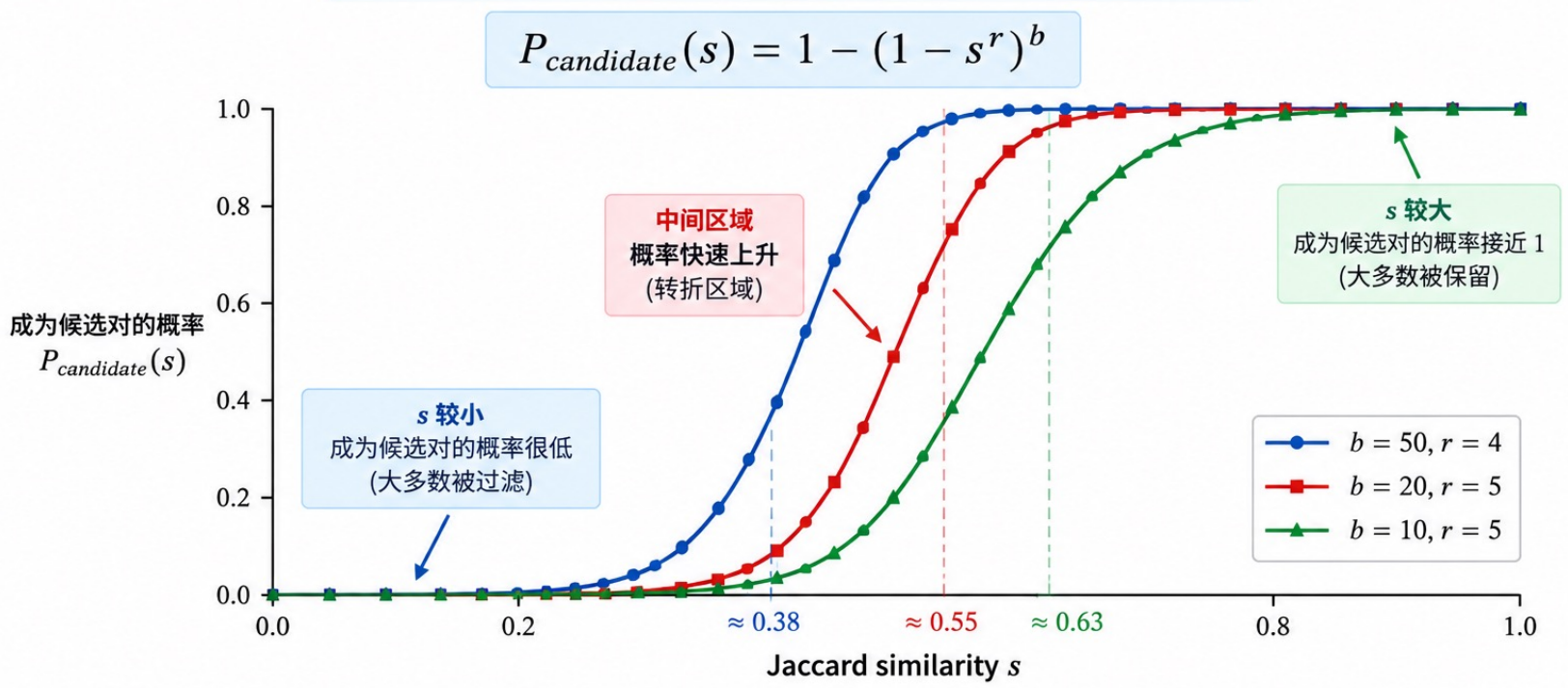
所以 A 和 B 会成为：Candidate Pair

然后再对 A 和 B 做真正的 Jaccard similarity 计算。

概率分析

- 假设 A, B的 Jaccard 相似度为 s，且各 MinHash 函数独立：$P[h_i(A) = h_i(B)] = s$
- 一个 band 中 r行全部相同：$P = s^r$
- b个 band 都不相同：$P = (1 - s^r)^b$
- 因此至少一个 band 完全相同、成为 Candidate Pair 的概率：$\boxed{P_{\text{candidate}}(s) = 1 - (1 - s^r)^b}$
- **Band 内 AND，Band 间 OR，从而放大相似与不相似对象之间的概率差异。**
  - s 高 $\Rightarrow P_{\text{candidate}} \approx 1$
  - s 低 $\Rightarrow P_{\text{candidate}} \approx 0$
- $r \uparrow \Rightarrow$ band 内条件更严格，false positive 更少，但更容易漏掉相似对象
- $b \uparrow \Rightarrow$ 尝试次数更多，recall 更高，但候选对也更多

S-Curve



- **参数影响：** r 越大 → 曲线越右移（更严格，false positive 更少，但更容易漏掉相似对象）
  b 越大 → 曲线越陡（尝试次数更多，recall 更高，但候选对也更多）
- **粗略阈值：** $s^* \approx \left(\frac{1}{b}\right)^{1/r}$ （给出 S-curve 转折位置的近似）
- **注意：** 这不是一个硬阈值，而是概率上的筛选。

核心直觉：
低相似度的对象很难成为候选，
高相似度的对象很容易成为候选，
通过选择 b 和 r，可以在某个相似度阈值附近实现有效的区分。

LSH 一般化

- 对任意距离函数 D，只要构造一个哈希族满足：
- $D(x,y) \le d_1 \Rightarrow \Pr[h(x)=h(y)] \ge p_1$
- $D(x,y) \ge d_2 \Rightarrow \Pr[h(x)=h(y)] \le p_2$
- 就称它为 LSH family。

MinHash 是 Jaccard distance 下的一种 LSH family

1. Jaccard 相似度与距离

$$J(A,B)=\frac{|A\cap B|}{|A\cup B|}$$

$$D_J(A,B)=1-J(A,B)$$

$0 \le J(A,B) \le 1$

$0 \le D_J(A,B) \le 1$

2. MinHash 的基本性质

$$\Pr[h(A)=h(B)]=J(A,B)=1-D_J(A,B)$$

3. 当距离较小时（相似对象）

若 $D_J(A,B) \le d_1$，则 $J(A,B) \ge 1-d_1$，
因此 $Pr[h(A)=h(B)] \ge 1-d_1$.

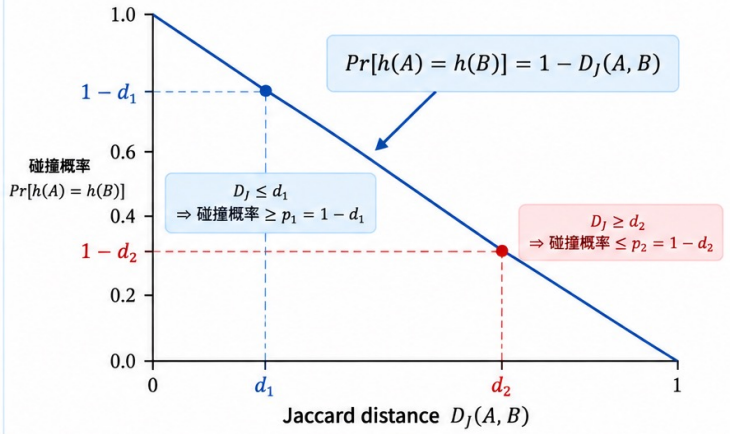
可以取 $p_1 = 1-d_1$.

4. 当距离较大时（不相似对象）

若 $D_J(A,B) \ge d_2$，则 $J(A,B) \le 1-d_2$，
因此 $Pr[h(A)=h(B)] \le 1-d_2$.

可以取 $p_2 = 1-d_2$.

MinHash 的碰撞概率与 Jaccard distance 的关系



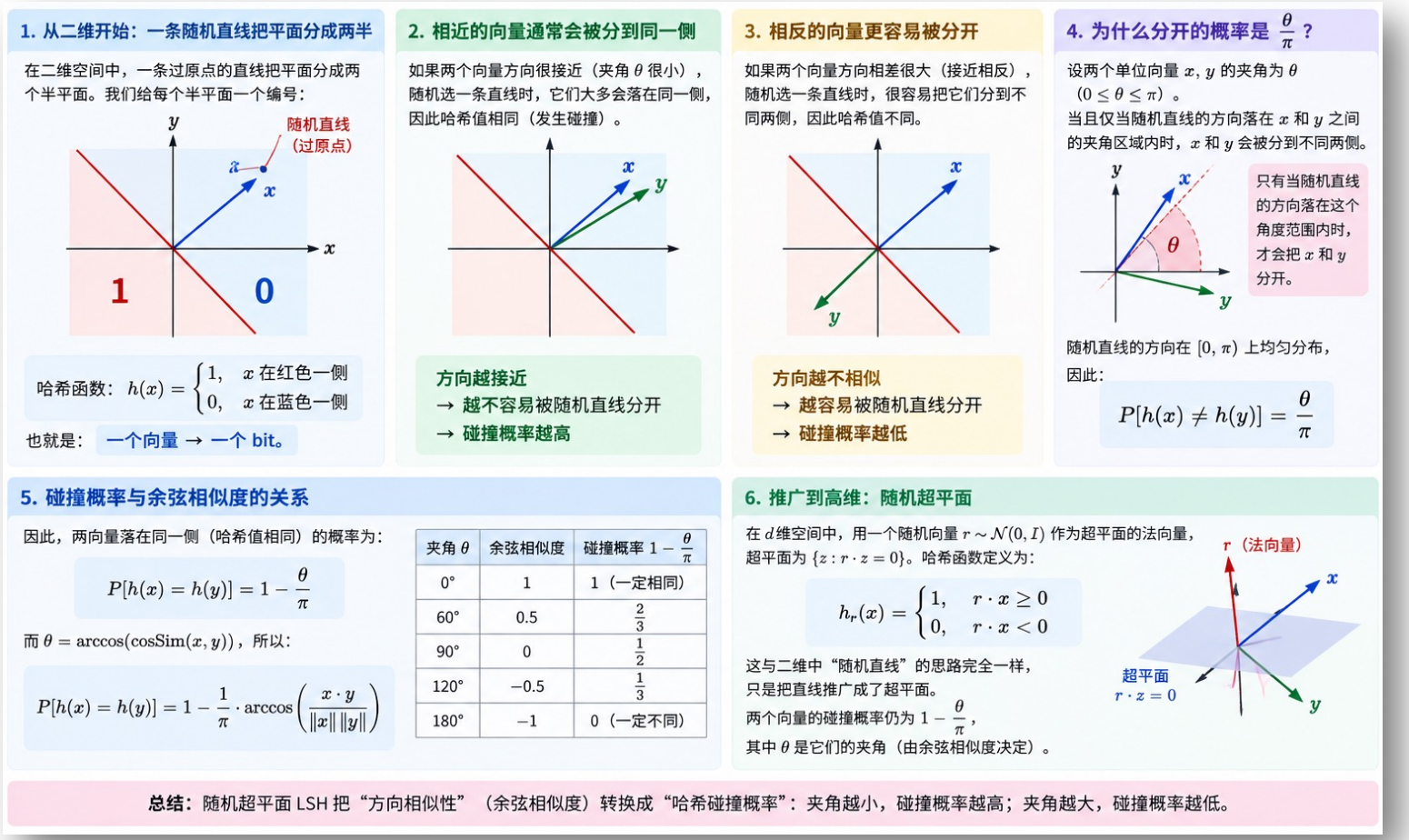
结论

MinHash 满足在 Jaccard distance 下的 (d_1, d_2, p_1, p_2)-sensitive 条件：

$$p_1 = 1-d_1, \qquad p_2 = 1-d_2$$

因此，MinHash 是 Jaccard distance 下的一种 LSH family.

用随机超平面做LSH



单个随机超平面只给你 1 bit：

$$h(x) \in \{0,1\}$$

这个太粗，所以一般不是用一个 hash，而是用 k 个随机超平面拼成一个签名：

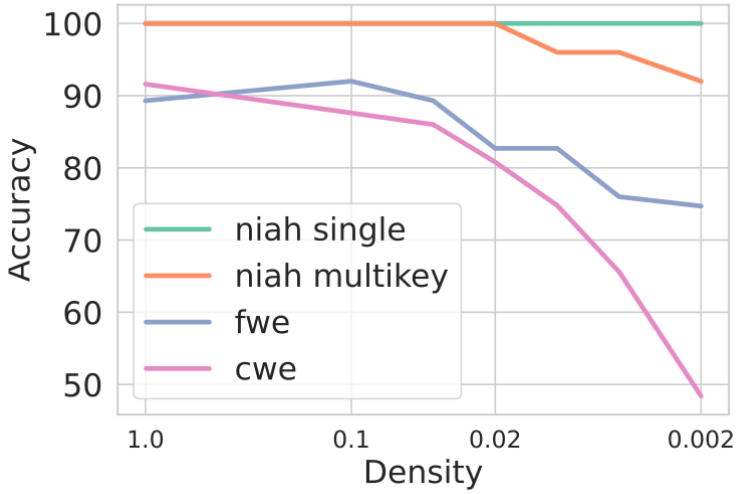
$$g(x) = \left(h_1(x), h_2(x), \ldots, h_k(x)\right)$$

- 分L个表，每个表1个签名
- 任意一个表的签名在同一个桶
- 再做具体的Cosine Similarity计算

随机画一条过原点的超平面，看两个向量落在超平面的同一侧还是不同侧。

应用：Transformer模型

- 标准 Attention：$o = \sum_{i=1}^{n} w_i \, v_i$；其中$w_i = \frac{e^{qk_i^T/\sqrt{d}}}{\sum_j e^{qk_j^T/\sqrt{d}}}$.
- 如果上下文有 $100K$ token，每生成一个新 token，都要计算：$qk_1^T, qk_2^T, \ldots, qk_{100K}^T$.



横轴表示：TopK Attention 所使用的 Attention Keys 占全部 Keys 的比例。

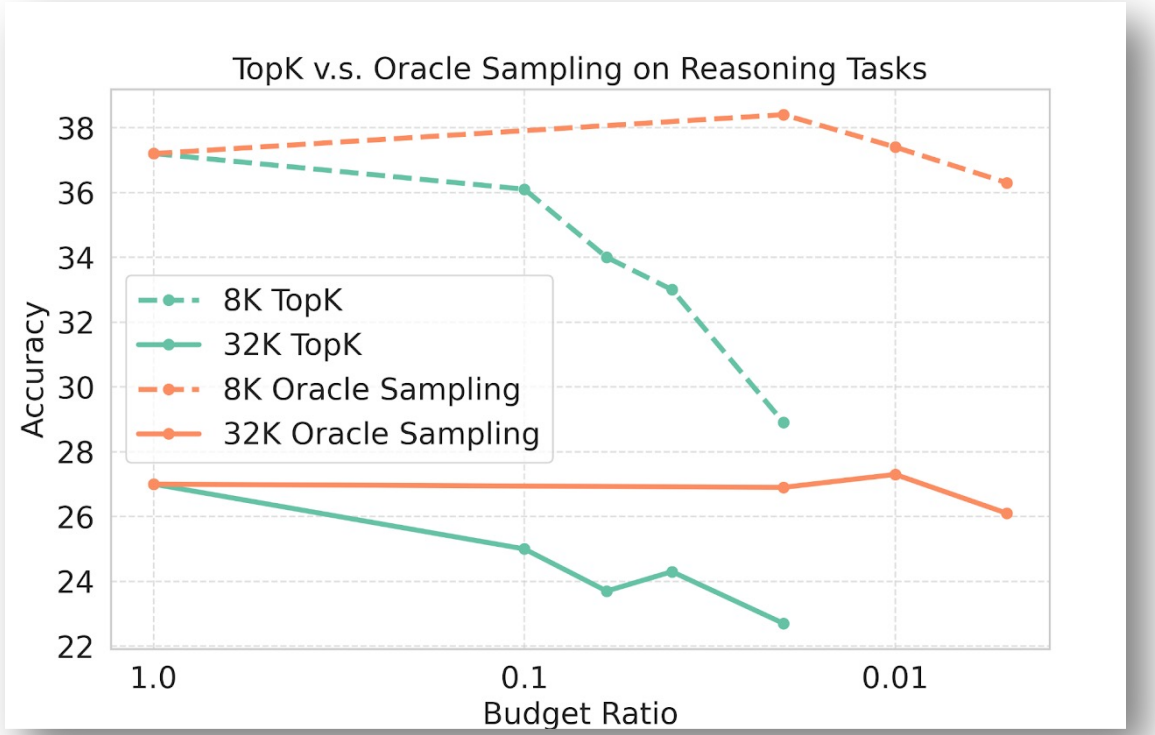
Attention 很稀疏，那我只保留最大的 Top-K 不就好了？

Attention 并不总是稀疏到可以安全地直接砍掉剩余部分；
TopK 对某些任务会造成明显质量损失

- TopK Attention 在检索类任务（如 NIAH）上表现较好，因为这类任务中的有效信息通常集中在少数几个词上；
- 但在需要聚合大量上下文信息的任务（如 CWE、FWE）中，其性能会显著下降。

第一个想法：别 **Top-K**，改成 **Sampling**

- 注意力其实可以理解成一个期望：$\boxed{o = \sum_i w_i \, v_i = \mathbb{E}_{i\sim w}[v_i]}$，这就非常有意思了。
- 假设：$w_1 = 0.4, w_2 = 0.3, w_3 = 0.1, \ldots$，那么不一定非得把所有$v_i$都算一遍。
- 可以按照：w_i来抽样。
  - Attention 大的 token：$\boxed{\text{多抽}}$
  - Attention 小的 token：$\boxed{\text{少抽，但不是直接扔掉}}$
- 然后通过采样估计整个 attention output。



问题

- 想按照$w_i \propto e^{qk_i^T/\sqrt{d}}$进行 sampling。
- 问题是：**我要知道 w_i，不还是得先把所有 qk_i^T 算出来吗？**
- 那就又回到了：$qk_1, qk_2, \ldots, qk_n$ 全算一遍。

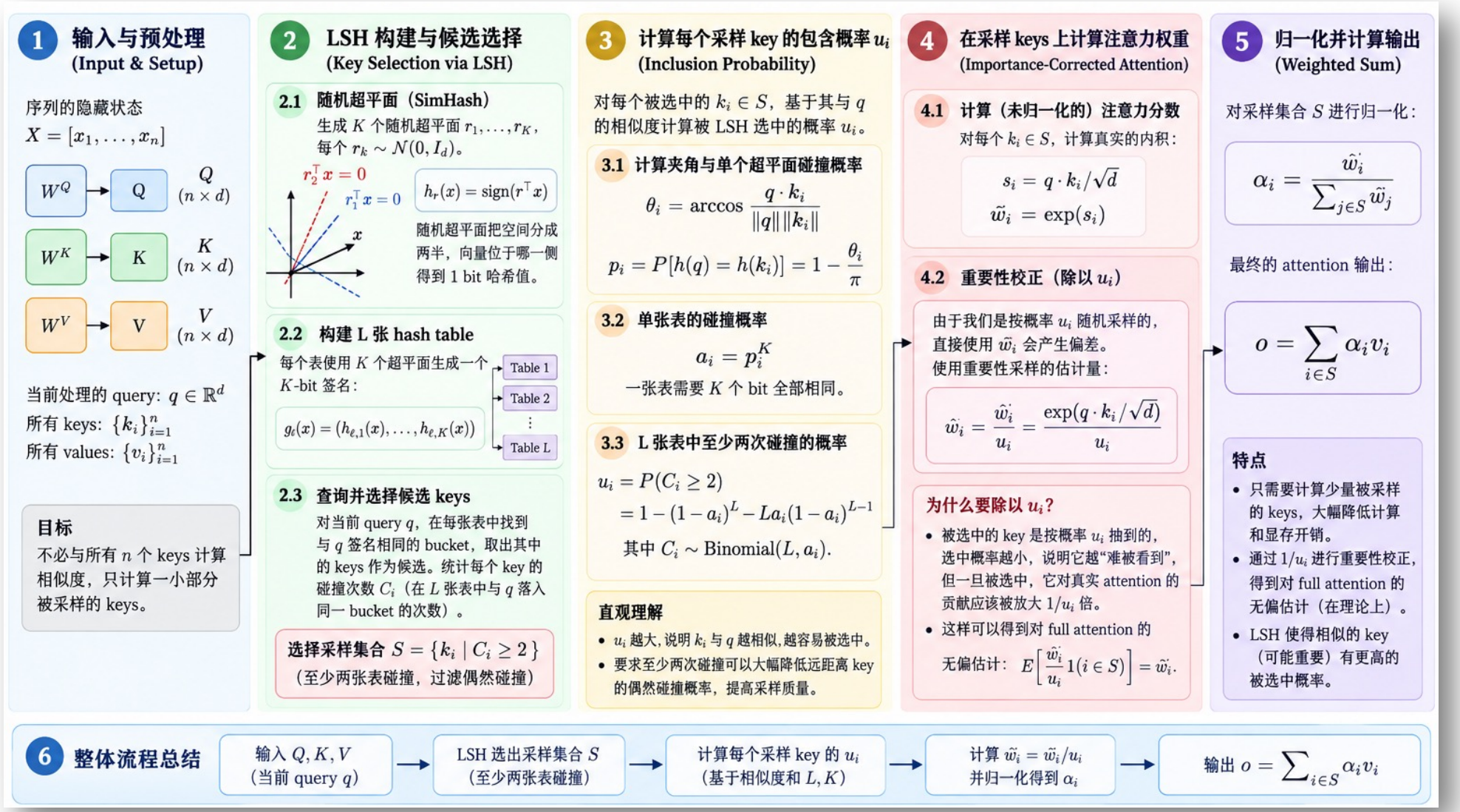
- 问题：

如何不计算所有 qk_i^T，却知道哪些 k_i 应该更容易被采样？

方法

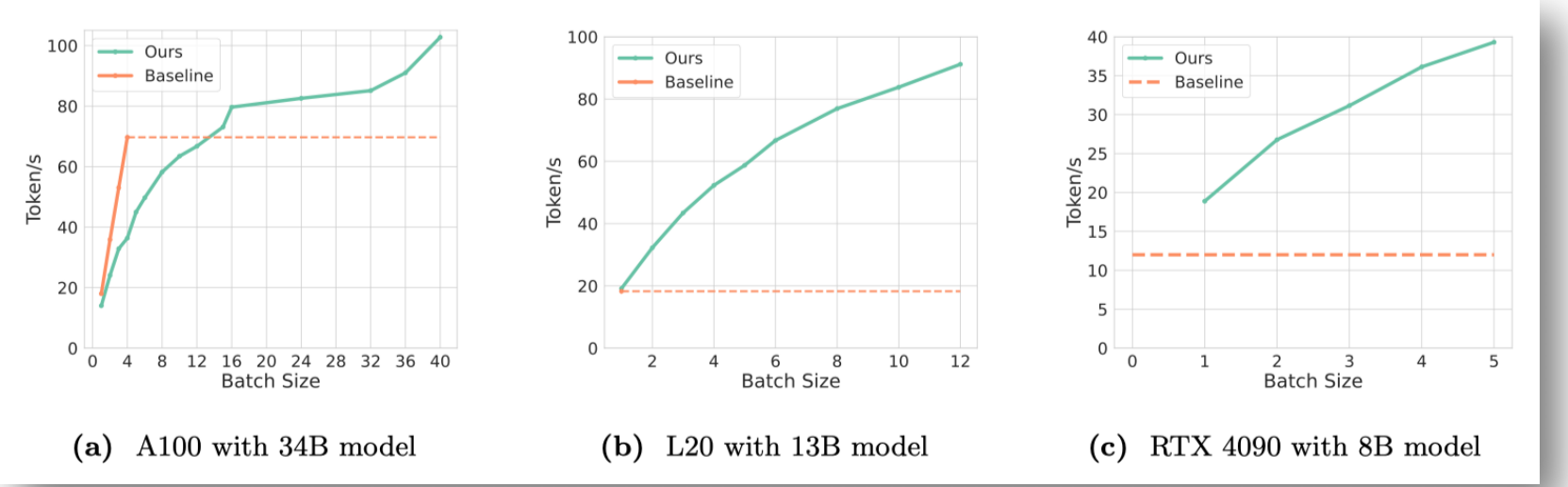
- 利用Random Hyperplane LSH：q 和 k_i 越相似 ⇒ k_i 越容易和 q LSH 碰撞
- 而 attention 又是：$w_i \propto e^{qk_i^T/\sqrt{d}}$；$qk_i^T$ 越大：Attention 越大。
- 所以 有个对应：qk_i^T 大 ⇒ Attention 大
- 同时：q, k_i 相似 ⇒ LSH collision probability 大
- 因此可以用：LSH collision probability 做代理，近似告诉我们：哪些 Key 应该以更高概率被采样

方法


1 输入与预处理 (Input & Setup)
序列的隐藏状态 $X = [x_1, \ldots, x_n]$
W^Q Q Q $(n \times d)$
W^K K K $(n \times d)$
W^V V V $(n \times d)$
当前处理的 query: $q \in \mathbb{R}^d$
所有 keys: $\{k_i\}_{i=1}^n$
所有 values: $\{v_i\}_{i=1}^n$
目标
不必与所有 n 个 keys 计算相似度，只计算一小部分被采样的 keys。
2 LSH 构建与候选选择 (Key Selection via LSH)
2.1 随机超平面（SimHash）
生成 K 个随机超平面 $r_1, \ldots, r_K$，每个 $r_k \sim \mathcal{N}(0, I_d)$。
$r_2^\top x = 0$
$r_1^\top x = 0$
x
$h_r(x) = \text{sign}(r^\top x)$
随机超平面把空间分成两半，向量位于哪一侧得到 1 bit 哈希值。
2.2 构建 L 张 hash table
每个表使用 K 个超平面生成一个 K-bit 签名：
$g_\ell(x) = (h_{\ell,1}(x), \ldots, h_{\ell,K}(x))$
Table 1
Table 2
Table L
2.3 查询并选择候选 keys
对当前 query q，在每张表中找到与 q 签名相同的 bucket，取出其中的 keys 作为候选。统计每个 key 的碰撞次数 C_i（在 L 张表中与 q 落入同一 bucket 的次数）。
选择采样集合 $S = \{k_i \mid C_i \geq 2\}$
（至少两张表碰撞，过滤偶然碰撞）
3 计算每个采样 key 的包含概率 u_i (Inclusion Probability)
对每个被选中的 $k_i \in S$，基于其与 q 的相似度计算被 LSH 选中的概率 u_i。
3.1 计算夹角与单个超平面碰撞概率
$\theta_i = \arccos \frac{q \cdot k_i}{\|q\| \|k_i\|}$
$p_i = P[h(q) = h(k_i)] = 1 - \frac{\theta_i}{\pi}$
3.2 单张表的碰撞概率
$a_i = p_i^K$
一张表需要 K 个 bit 全部相同。
3.3 L 张表中至少两次碰撞的概率
$u_i = P(C_i \geq 2)$
$= 1 - (1 - a_i)^L - L a_i (1 - a_i)^{L-1}$
其中 $C_i \sim \text{Binomial}(L, a_i)$.
直观理解
• u_i 越大，说明 k_i 与 q 越相似，越容易被选中。
• 要求至少两次碰撞可以大幅降低远距离 key 的偶然碰撞概率，提高采样质量。
4 在采样 keys 上计算注意力权重 (Importance-Corrected Attention)
4.1 计算（未归一化的）注意力分数
对每个 $k_i \in S$，计算真实的内积：
$s_i = q \cdot k_i / \sqrt{d}$
$\tilde{w}_i = \exp(s_i)$
4.2 重要性校正（除以 u_i）
由于我们是按概率 u_i 随机采样的，直接使用 $\tilde{w}_i$ 会产生偏差。
使用重要性采样的估计量：
$\hat{w}_i = \frac{\tilde{w}_i}{u_i} = \frac{\exp(q \cdot k_i / \sqrt{d})}{u_i}$
为什么要除以 u_i？
• 被选中的 key 是按概率 u_i 抽到的，选中概率越小，说明它越“难被看到”，但一旦被选中，它对真实 attention 的贡献应该被放大 $1/u_i$ 倍。
• 这样可以得到对 full attention 的无偏估计：$E\left[\frac{\tilde{w}_i}{u_i} 1(i \in S)\right] = \tilde{w}_i$.
5 归一化并计算输出 (Weighted Sum)
对采样集合 S 进行归一化：
$\alpha_i = \frac{\hat{w}_i}{\sum_{j \in S} \hat{w}_j}$
最终的 attention 输出：
$o = \sum_{i \in S} \alpha_i v_i$
特点
• 只需要计算少量被采样的 keys，大幅降低计算和显存开销。
• 通过 $1/u_i$ 进行重要性校正，得到对 full attention 的无偏估计（在理论上）。
• LSH 使得相似的 key（可能重要）有更高的被选中概率。
6 整体流程总结
输入 Q, K, V （当前 query q）
LSH 选出采样集合 S （至少两张表碰撞）
计算每个采样 key 的 u_i （基于相似度和 L, K）
计算 $\hat{w}_i = \tilde{w}_i / u_i$ 并归一化得到 α_i
输出 $o = \sum_{i \in S} \alpha_i v_i$

实验结果

- 在 A100 GPU 上服务 34B 模型、上下文长度为 16K 时，MagicPIG 将吞吐量提升了 **1.5**×。
- 在 L20 GPU 上服务 13B 模型、上下文长度为 16K 时，MagicPIG 将吞吐量提升了 **5.0**×。
- 在模拟的 RTX 4090 环境下服务 8B 模型、上下文长度为 96K 时，MagicPIG 在单请求服务场景下实现了 **54 ms** 的延迟，并可将基线方法的吞吐量最高提升 **3.3**×。
- 图中的虚线表示各基线方法能够达到的最高吞吐量。通过将 KV Cache 卸载（offloading）到 CPU， 相比完全在 GPU 上执行 Full Attention，可以支持更大的 batch size，这也是其吞吐量提升的重要原因。



参考文献：Chen et al., “MagicPIG: LSH Sampling for Efficient LLM Generation,” ICLR 2025.

大作业

- 发布在 elearning 上，通过 elearning 提交（有deadline）